
答卷编号：

论文题目：风电功率预测问题

	姓 名	专业、班级	有效联系电话
参赛队员 1	周小陈	工程力学	15210617266
参赛队员 2	刘真畅	飞行器设计与工程	13651330581
参赛队员 3	张宇飞	飞行器设计与工程	15601142560

指导教师：金海

参赛学校：北京理工大学

报名序号：1550

证书邮寄地址：北京理工大学中关村校区 徐厚宝

（学校统一组织的请填写负责人）

答卷编号：

	阅卷专家 1	阅卷专家 2	阅卷专家 3
论文等级			

风电功率预测问题

摘要:

本文着力研究了风电功率的预测问题。根据相关要求，本文中我们分别利用 ARMA 模型、卡尔曼滤波预测模型和小波神经网络预测模型对该风电场的风电功率进行预测。通过对预测结果各项评价指标的综合分析，发现：小波神经网络预测模型的精确度最高；单台风电机组预测误差与总机组预测误差成正相关性；多个风电机组的汇聚会使得总体的预测误差减小。另外，从神经网络的训练过程中，我们发现突加扰动是阻碍风电功率实时预测精度进一步改善的主要因素，风电功率的预测精度不可能无限提高。

对于问题一，我们分别建立了 ARMA、卡尔曼滤波、小波神经网络三种预测模型对指定的发电机组的输出功率进行了预测，取得了较为理想的结果。ARMA 模型的预测精确度为 75.4%—79.3%，卡尔曼滤波模型的预测精确度为 81.3%—95%，小波神经网络模型的预测精确度为 92.1%—94.7%，故小波神经网络的预测效果最好。

对于问题二，我们分析比较了三种模型下单台机组和多机组 5 月 21 日至 6 月 6 日的平均相对预测误差，得知风电机组的汇聚会使得总体的预测误差减小。

针对问题三，我们在问题一小波神经网络模型的基础上建立了遗传神经网络模型。经过仿真，我们发现该模型能显著减小峰值误差，有力地抑制时间延迟现象，有效地提高了预测的精确度。对仿真误差进行分析，我们指出突加的扰动是阻碍风电功率实时预测精度进一步改善的主要因素，预测的精度不可能无限提高。

关键词：ARMA，卡尔曼滤波，小波神经网络，遗传神经网络

一、问题重述

随着科学技术的发展,风力发电技术也得到快速发展。因为风力具有波动性、间歇性、能量密度低等特点,风电功率也是波动的。大规模风电场接入电网运行时,大幅度地风电功率波动会对电网的功率平衡和频率调节带来不利影响。因此,如何对风电场的发电功率进行尽可能准确的预测是急需解决的问题。本文在某风电场 58 台风电机组输出功率数据的基础上,需解决以下问题:

- (1) 至少采用三种预测方法对给定的数据进行风电功率实时预测并检验预测结果是否满足预测精度的相关要求。
- (2) 比较单台风电机组功率的相对预测误差与多机总功率的相对预测误差,分析风电机组的汇聚对于预测结果误差的影响,并做出预期。
- (3) 在问题(1)的基础上,构建有更高预测精度的实时预测方法,并用预测结果说明其有效性。
- (4) 在以上问题的基础上,分析论证阻碍风电功率实时预测精度进一步改善的主要因素。判断风电预测精度能否无限提高。

二、问题分析

本题是一个预测类问题,它以风力发电为背景,主要考察对于风电发电功率进行预测的能力。

首先,被预测量是随时间变化的序列,被预测量随时间的变化规律具有很强的非线性,因此我们采用的算法不仅要能够对时间序列进行预测,还必须具备一定的非线性处理能力。

针对问题一,我们建立三种模型,可以得到模型的预测结果。我们根据所给定的考核要求,能够计算得到模型的准确性。我们以准确性作为主要的评判标准,给出我们推荐的模型。

在问题一中,我们已经得到了单台风电机组与多台发电机组功率的预测误差。进一步处理,我们可以给出单台发电机组与多台发电机组的相对误差。我们对所得相对误差数据进行统计分析,可以得到

三、模型假设

- (1) 观测数据真实可靠
- (2) 短期内不存在大的自然灾害,例如地震、海啸以及台风等等
- (3) 预测期间风电机组分布不变,发电机组性能不随时间发生变化

四、参数说明

L ——滞后延迟算子	$h(j)$ ——隐含层第 j 个节点输出值
y_t ——风电功率的时间序列	ω_{ij} ——输入层和隐含层的连续权值
p ——自回归的阶数	b_j ——小波基函数的平移因子
ε_t ——零均值的系统白噪声	a_j ——小波基函数 h_j 的伸缩因子
q ——移动平均的阶数	h_j ——小波基函数
$MSPE$ ——均方百分比误差	$h(i)$ ——第 i 个隐含层节点的输出
Cap ——风电场的开机容量	l ——隐含层节点数
$MAPE$ ——平均百分比误差	m ——输出层节点数
r_1 ——精确度	$yn(k)$ ——期望输出
r_2 ——合格率	$y(k)$ ——小波神经网络预测输出
P_{Mk} —— k 时段的实际平均功率	η ——学习效率
P_{Pk} —— k 时段的预测平均功率	y_i ——BP 神经网络第 i 个节点的期望输出
N ——日考核总时段数	o_i ——BP 神经网络第 i 个节点的预测输出
I_{m-1} ——状态空间模型的自回归系数	$a_{\max}$ ——基因 a_{ij} 的上界
$X_1, X_2, \dots, X_k$ ——小波神经网络的输入参数	$a_{\min}$ ——基因 a_{ij} 的下界
$Y_1, Y_2, \dots, Y_m$ ——小波神经网络的预测输出	g ——当前迭代次数
ω_{ij}, ω_{jk} ——小波神经网络权值	$G_{\max}$ ——最大进化次数

五、模型建立

1. 风电功率实时预测及误差分析

目前, 风电功率预测的方法主要有持续预测法、时间序列法(包括 AR、MA、ARMA、ARIMA 等)、神经网络法(ANN)、小波分析法、支持向量机法(SVM)等。综合考虑风电功率的随机性特征和各算法的优缺点, 我们选择了 ARMA 法、卡尔

曼滤波法和小波神经网络等三种方法对风电功率进行了预测。

1.1. ARMA 预测模型

1.1.1. ARMA 模型的基本原理

ARMA 模型是常用的时间序列模型，其基本的类型为：

(1) 自回归 (AR) 模型。 $AR(p)$ 为

$$(L)y_t = \varepsilon_t \quad (1)$$

其中， L 为滞后延迟算子； y_t 为风电功率的时间序列； $Ly_t = y_{t-1}$ ； p 为自回归的阶数； ε_t 为零均值的系统白噪声。

(2) 滑动平均 (MA) 模型。 $MA(q)$ 为

$$y(t) = \theta(L)\varepsilon_t \quad (2)$$

其中， q 为移动平均的阶数。

(3) ARMA 模型。 $ARMA(p, q)$ 为

$$\varphi(L)y_t = \theta(L)\varepsilon_t \quad (3)$$

由以上三式可见，AR 模型和 MA 模型可视为 ARMA 模型的特殊情况。ARMA 模型的平稳条件是滞后多项式 $\varphi(L)$ 的根在单位圆外，可逆条件为 $\theta(L)$ 的根都在单位圆外。ARMA 模型对数据平稳性有要求，要在平稳时间序列的大前提下建模，所以要用 ARMA 模型预测风电功率，首先要检验风电功率时间序列的平稳性。时间序列平稳性检验常用的方法为增广 Dickey-Fuller (ADF) 检验，ADF 检验包括一个回归方程：

$$\nabla y_t = \beta_1 y_{t-1} + c_1 \nabla y_{t-1} + c_2 \nabla y_{t-2} + \cdots + c_{p-1} \nabla y_{t-p+1} + \varepsilon_t + \beta_2 t \quad (4)$$

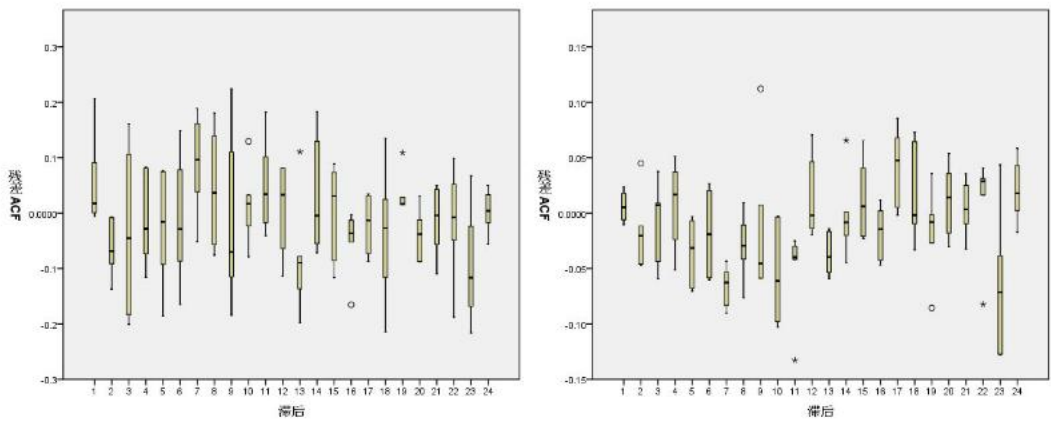
上式左边为序列的一阶差分项，右边为序列的一阶滞后项、滞后差分项，有时还有常数项和时间趋势项。在进行 ADF 检验时，需根据实际情况选择回归中是否包括常数项、线性时间趋势及回归中的滞后阶数 p 的选择可根据保证 ε_t 是白噪声过程的最小 p 值的标准进行选择。在每种情况下，单位根检验都对回归式中 y_{t-1}

的系数进行检验，如果系数显著不为零，那么 y_t 包含单位根的假设将被拒绝， y_t 序列即是平稳的。

1.1.2. 平稳性检验

我们取该风电场 2006 年 5 月 10 日至 6 月 6 日共 28 天的风电功率实测数据作为研究对象，以其中前 21 天地风电功率数据建立模型。首先采用 ADF 及 ACF 检验来检验该时间序列的平稳性：如该风电功率时间序列是平稳的，则满足 ARMA 模型前提；如该序列不平稳，则对差分后序列建立 ARMA 模型，如仍不平稳，则继续做差分，直到差分后序列平稳，ARMA 建模前提满足为止。

各风电机组的 ACF 检验结果如下图所示：



图（1）a 时间段机组 ACF 图 图（2）b 时间段机组 ACF 图

各风电机组的 ADF 检验结果见表 1。

	ADF 检验统计量	1%临界值	5%临界值	10%临界值
机组 A	-4.091682	-3.433938	-2.863011	-2.567601
机组 B	-5.830311	-3.440688	-2.865984	-2.569195
机组 C	-4.835924	-3.440973	-2.864253	-2.567613
机组 D	-4.257462	-3.437082	-2.867812	-2.567915
四台机组	-5.648925	-3.482525	-2.864214	-2.598445
58 台机组	-4.956412	-3.459961	-2.857145	-2.584562

表 1 ADF 检验结果

比较 ADF 检验统计量与临界值大小，可判断时间序列是否平稳。由表 1 可见，以上六种情况的风电功率时间序列 ADF 检验统计量均小于 1%临界值的显著水平，所以，在 95%置信水平下有理由拒绝原假设，即本序列是平稳的，满足 ARMA 建模的前提条件，因此，可考虑将风电功率时间序列 y_t 识别为 $ARMA(p, q)$ 结构。

1.1.3. 建立 ARMA 模型

鉴于模型 $ARMA(p, q)$ 的识别具有很大的灵活性，为了得到最合理的模型，

本文采取了定阶步骤，根据时间序列的自相关、偏相关函数分析图，对多组可行阶数进行了参数估计，对所有备选模型进行模型诊断，筛选出备选模型集。由于许瓦兹信息准则 SIC 的强一致性，在理论层面上能够渐进地选择真实模型，所以计算备选模型集中所有模型的 SIC。考虑模型的可逆性和稳定性条件，得到数据样本的 ARMA 模型的参数如表 2。

	机组 A	机组 B	机组 C	机组 D	四机组	58 机组
p	2	2	2	2	2	2
q	2	2	2	2	2	1

表 2 ARMA 模型参数

依照经典时间序列分析的步骤，在完成模型阶数识别后，使用极大似然估计法获得模型的参数估计模型分别为：

$$P_A: \hat{y}_t = 269.4057 - 1.8881 \hat{y}_{t-1} + 0.88844 \hat{y}_{t-2} - 1.2765 \hat{\varepsilon}_{t-1} + 0.30239 \hat{\varepsilon}_{t-2} \quad (5)$$

$$P_B: \hat{y}_t = 231.7165 - 1.8762 \hat{y}_{t-1} + 0.87663 \hat{y}_{t-2} - 1.2953 \hat{\varepsilon}_{t-1} + 0.32144 \hat{\varepsilon}_{t-2} \quad (6)$$

$$P_C: \hat{y}_t = 222.7115 - 1.8868 \hat{y}_{t-1} + 0.88712 \hat{y}_{t-2} - 1.2855 \hat{\varepsilon}_{t-1} + 0.30922 \hat{\varepsilon}_{t-2} \quad (7)$$

$$P_D: \hat{y}_t = 236.1261 - 1.8818 \hat{y}_{t-1} + 0.88224 \hat{y}_{t-2} - 1.2782 \hat{\varepsilon}_{t-1} + 0.30509 \hat{\varepsilon}_{t-2} \quad (8)$$

$$P_4: \hat{y}_t = 959.9598 - 1.8937 \hat{y}_{t-1} + 0.89401 \hat{y}_{t-2} - 1.0767 \hat{\varepsilon}_{t-1} + 0.10732 \hat{\varepsilon}_{t-2} \quad (9)$$

$$P_{58}: \hat{y}_t = 12265 - 1.9013 \hat{y}_{t-1} + 0.90162 \hat{y}_{t-2} - 0.9674 \hat{\varepsilon}_{t-1} \quad (10)$$

1. 1. 4. 预测结果及误差分析

运用 ARMA 模型分别对 5 月 31 日 0 时 0 分至 5 月 31 日 23 时 45 分（记 a 时域）、5 月 31 日 0 时 0 分至 6 月 6 日 23 时 45 分（记 b 时域）的 $P_A, P_B, P_C, P_D, P_4, P_{58}$ 进行预测，得到原始风电功率和预测风电功率。预测结果如下图所示。

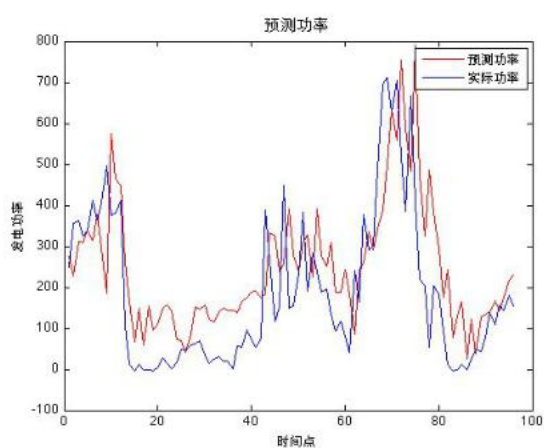


图 (3) a 时段 P_A 功率预测曲线

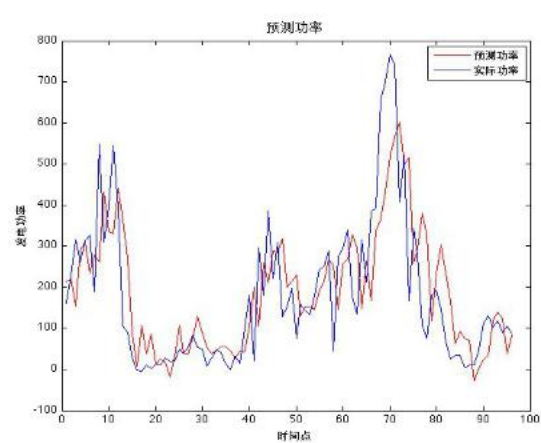


图 (4) a 时段 P_D 功率预测曲线

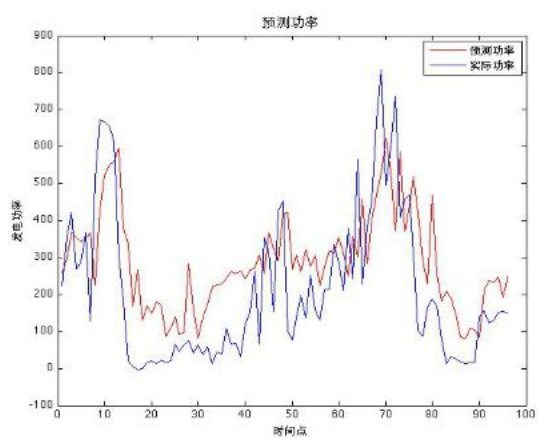


图 (5) a 时段 P_B 功率预测曲线

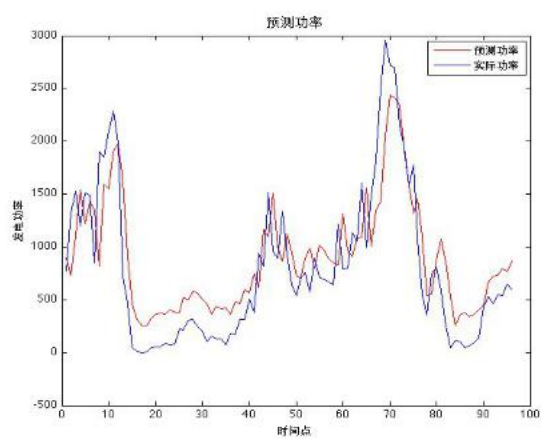


图 (6) a 时段 P_4 功率预测曲线

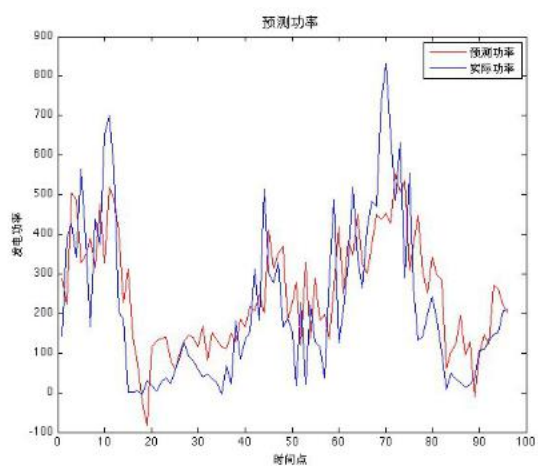


图 (7) a 时段 P_C 功率预测曲线

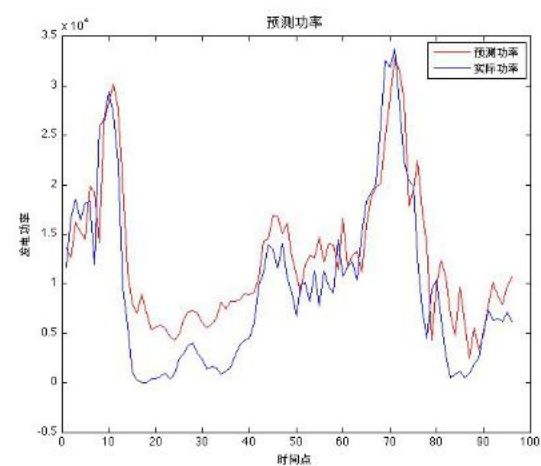
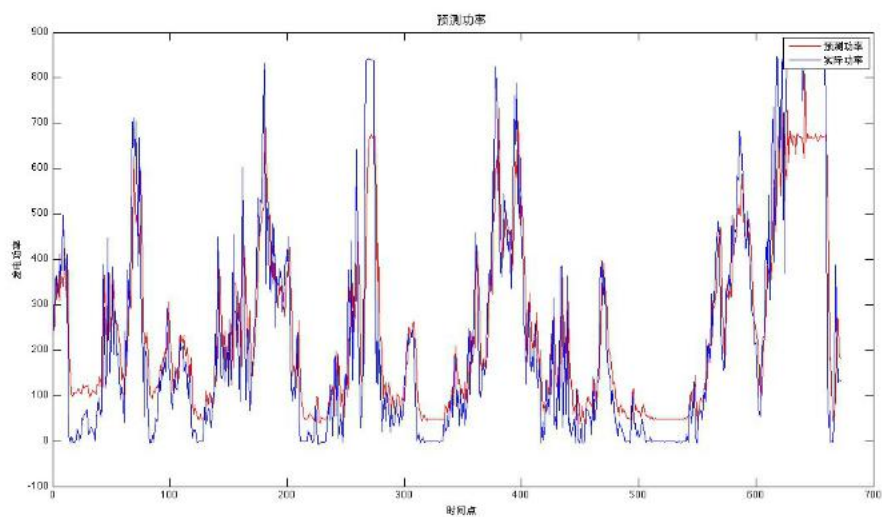
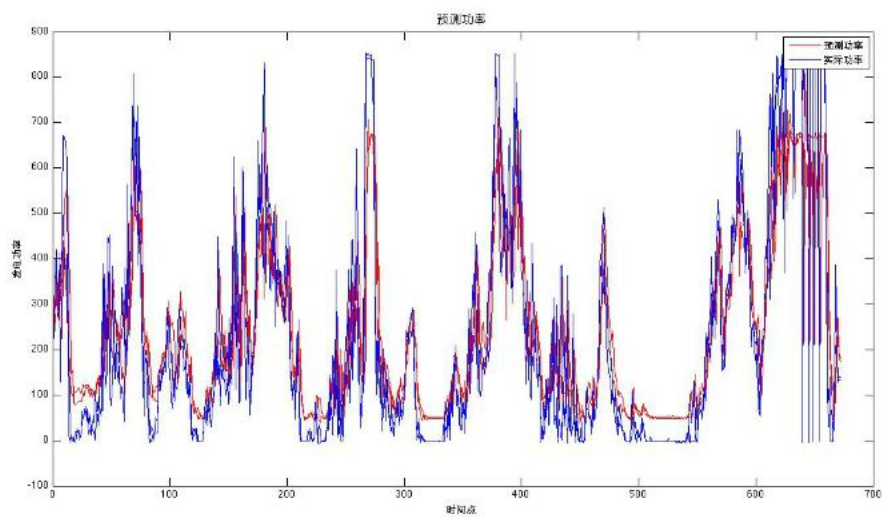


图 (8) a 时段 P_{58} 功率预测曲线



图（9）b时段PA功率预测曲线



图（10）b时段PB功率预测曲线

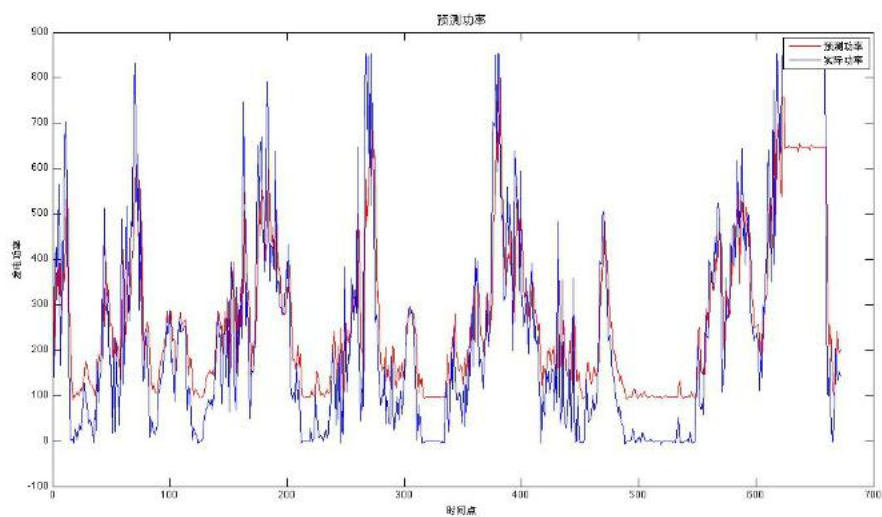


图 (11) b 时段 PC 功率预测曲线

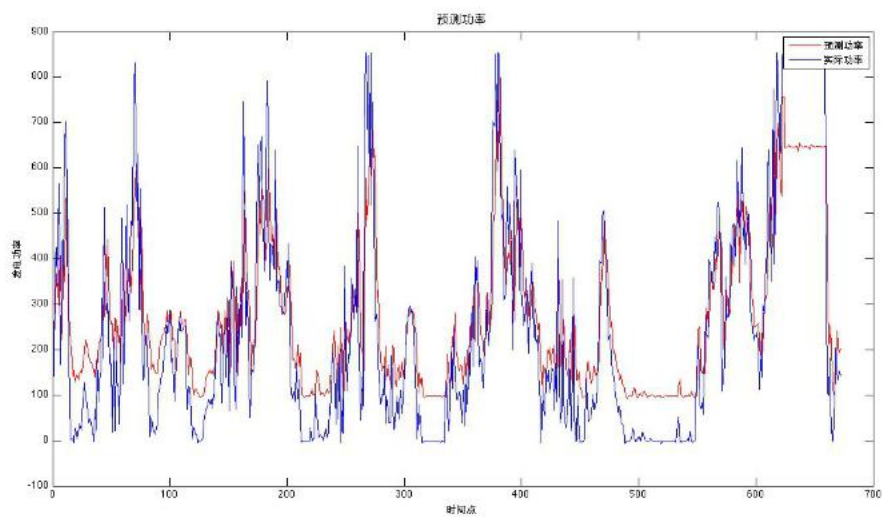
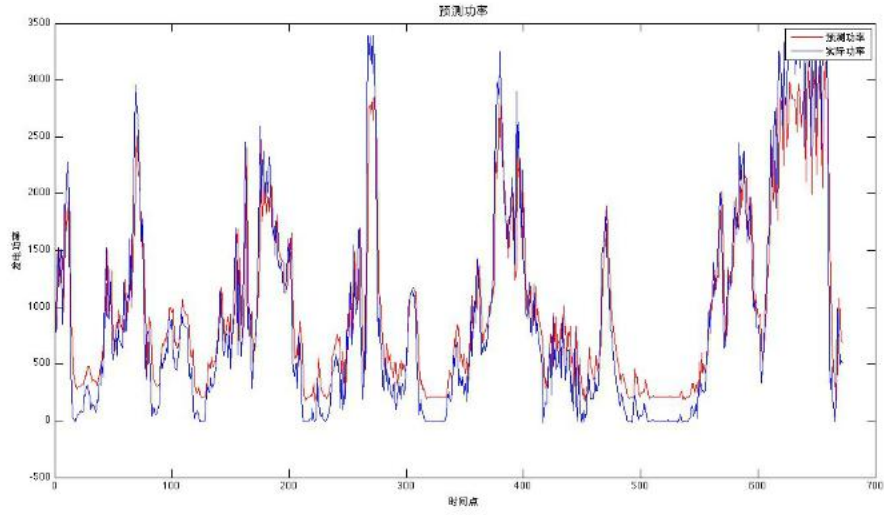
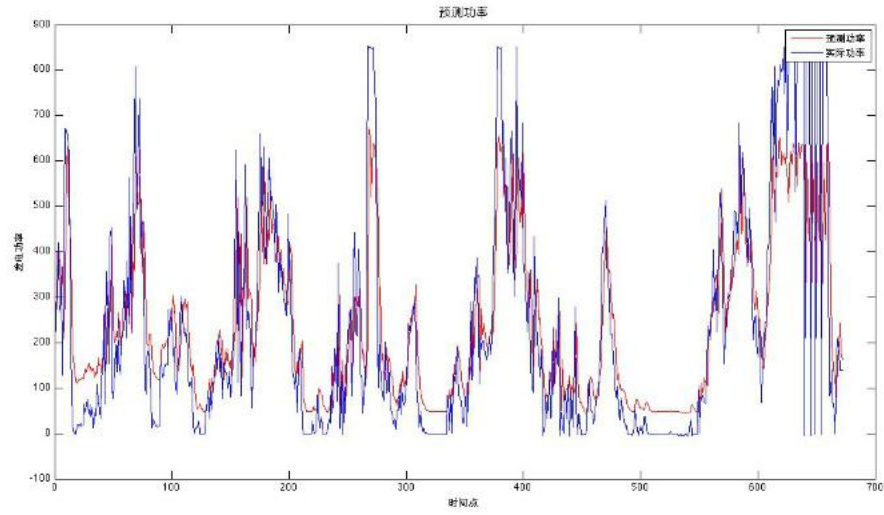


图 (12) b 时段 PD 功率预测曲线



图（13）b 时段 P4 功率预测曲线



图（14）b 时段 P58 功率预测曲线

为了对预测结果的精度和可靠性进行评价、分析，我们采用以下判断指标对预测结果进行分析。

$$(1) \text{ 均方百分比误差: } MSPE = \frac{1}{N} \sum_{t=1}^N \left| \hat{y}_t - y_t \right| \quad (11)$$

$$(2) \text{ 平均相对误差: } MAPE = \frac{1}{N} \sum_{t=1}^n \left| \frac{\hat{y}_t - y_t}{y_t} \right| \times 100\% \quad (12)$$

$$(3) \text{ 精确度: } r_1 = \left[1 - \sqrt{\frac{1}{N} \sum_{k=1}^N \left(\frac{P_{Mk} - P_{Pk}}{Cap} \right)^2} \right] \times 100\% \quad (13)$$

其中, P_{Mk} 为 k 时段的实际平均功率; P_{Pk} 为 k 时段的预测平均功率; N 为日考核总时段数 (取 96 点-免考核点数); Cap 为风电场的开机容量。

$$(4) \text{ 合格率: } r_2 = \frac{1}{N} \sum_{k=1}^N B_k \times 100\% \quad (14)$$

其中

$$\left(1 - \frac{P_{Mk} - P_{Pk}}{Cap} \right) \times 100\% \geq 75 \times 100\%, B_k = 1 \quad (15)$$

$$\left(1 - \frac{P_{Mk} - P_{Pk}}{Cap} \right) \times 100\% < 75 \times 100\%, B_k = 0 \quad (16)$$

通过 Matlab 的计算, 我们得到各项指标结果如表 3, 表 4。

	P_A	P_B	P_C	P_D	P_4	P_{58}
均方百分比误差/%	6.68	1101.41	24.27	25.79	111.39	66.85
平均相对误差/%	57.07	4641.00	47.12	165.40	85.14	52.28
精确度/%	63.77	71.86	68.63	63.29	79.30	75.40
合格率/%	94.79	68.75	77.08	87.50	82.11	75.00

表 3 ARMA 模型时段 a 各项指标结果

	P_A	P_B	P_C	P_D	P_4	P_{58}
均方百分比误差/%	8.68	923.03	27.53	36.51	94.19	65.32
平均相对误差/%	73.15	3773.39	39.46	182.36	79.25	58.46
平均精确度/%	62.49	74.90	73.85	62.57	69.84	84.65
平均合格率/%	94.51	88.65	92.94	83.62	87.62	96.01

表 4 ARMA 模型时段 b 的各项指标结果

通过分析, 我们发现 ARMA 模型对于风电功率短期预测的趋势是好的。比较

a 时段和 b 时段的预测结果，a 时段的预测精度相对高些，这是因为预测时间越长，风电功率波动性越强，因而建立的 ARMA 模型更难以描述风电功率的变化规律。另外，比较 ARMA 模型预测功率曲线和实际曲线，在变化规律发生改变时，预测曲线常常不能提前变化，而是存在一个“时滞”，所以，虽然 ARMA 模型基本能捕捉到风电功率曲线的变化规律，但该预测存在明显的延时性，而且精度不高。

1.2. 卡尔曼滤波预测模型

1.2.1. 模型建立

卡尔曼滤波法是采用状态方程和观测方程组成的线性随机系统的状态空间模型来描述滤波器，并利用状态方程的递推性，按线性无偏最小均方差估计准则，采用递推算法对滤波器的状态变量做最佳估计，从而求得滤掉噪声的有用信号的最佳估计。卡尔曼滤波算法可用于滤波、预测和平滑方面。本文利用卡尔曼滤波法对风电功率进行预测。

要实现卡尔曼滤波法预测风电功率，首先必须推导出正确的状态方程和测量方程。因已通过时间序列分析建立了风电功率时间序列的 ARMA 模型，故可将 ARMA 模型转换到状态空间，建立卡尔曼滤波的状态方程和测量方程。

对于一般的 ARMA 模型，可设为 $ARMA(p, q)$ ，其方程为

$$y_t = u_1 y_{t-1} + \cdots + u_p y_{t-p} + \varepsilon_t + v_1 \varepsilon_{t-1} + \cdots + v_q \varepsilon_{t-q} \quad (17)$$

假定扰动项 ε_t 都是关于 t 的白噪声，为了更容易转换，首先将上式改写为

$$y_t = u_1 y_{t-1} + \cdots + u_m y_{t-m} + \varepsilon_t + v_1 \varepsilon_{t-1} + \cdots + v_{m-1} \varepsilon_{t-m+1} \quad (18)$$

其中， $m = \max(p, q+1)$ 。当 $i > p$ 时， $u_i = 0$ ；当 $i > q$ 时， $v_i = 0$ 。此时， y_t 是经过零均值化所得时间序列。式 (17)、(18) 为 $ARMA(m, m-1)$ 模型，可写成状态空间模型为

$$\begin{cases} X_t = UX_{t-1} + VE_t \\ Y_t = CX_t \end{cases} \quad (19)$$

其中， $X_t = (y_t, y_{t-1}, \cdots, y_{t-m+1})'$ ， $E_t = (\varepsilon_t, \varepsilon_{t-1}, \cdots, \varepsilon_{t-m+1})'$ ，

$$C = (1, 0, \cdots, 0), \quad U_{m \times m} = \begin{pmatrix} u^* & u_m \\ I_{m-1} & 0 \end{pmatrix}, \quad V = \begin{pmatrix} v^* \\ 0 \end{pmatrix},$$

且 $u^* = (u_1, u_2, \cdots, u_{m-1})$ ， $v^* = (v_1, v_2, \cdots, v_{m-1})$ ； I_{m-1} 为状态空间模型的自回归系数。

即状态方程为:

$$\begin{bmatrix} y_t \\ y_{t-1} \\ \vdots \\ y_{t-m+1} \end{bmatrix} = \begin{bmatrix} u_1 & u_2 & \cdots & u_{m-1} & u_m \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ \vdots & \ddots & \ddots & \ddots & 0 \\ 0 & \cdots & 0 & 1 & 0 \end{bmatrix} * \begin{bmatrix} y_{t-1} \\ y_{t-2} \\ \vdots \\ y_{t-m} \end{bmatrix} + \begin{bmatrix} 1 & v_1 & \cdots & v_{m-1} & v_{m-2} \\ 0 & 0 & \cdots & 0 & 0 \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & 0 \end{bmatrix} * \begin{bmatrix} \varepsilon_t \\ \varepsilon_{t-1} \\ \vdots \\ \varepsilon_{t-m+1} \end{bmatrix} \quad (20)$$

测量方程:

$$y_t = (1, 0, \cdots, 0) * (y_t, y_{t-1}, \cdots, y_{t-m+1})' \quad (21)$$

状态空间方程和测量方程已经确立，只要确定相关的初始状态 $y(0)$ 和 $P(0)$ ，就可以利用递推方程进行迭代预测，但在实践中很难准确掌握初始状态 $y(0)$ 和 $P(0)$ 。卡尔曼预测在递推中不断用新的信息对状态进行修正，所以当预测时间足够长，初始值 $y(0)$ 和 $P(0)$ 对预测的影响将会衰减为零。考虑到收敛的速度和参考工程习惯，取初始值 $y(0)=[10]$ 和 $P(0)=10*I$ 。取系统噪声和测量噪声的协方差矩阵为单位阵，应用 Matlab 软件实现卡尔曼波预测风电功率。

1.2.2. 卡尔曼滤波法预测结果

通过 Matlab 程序实现卡尔曼波预测算法，我们得到预测结果。

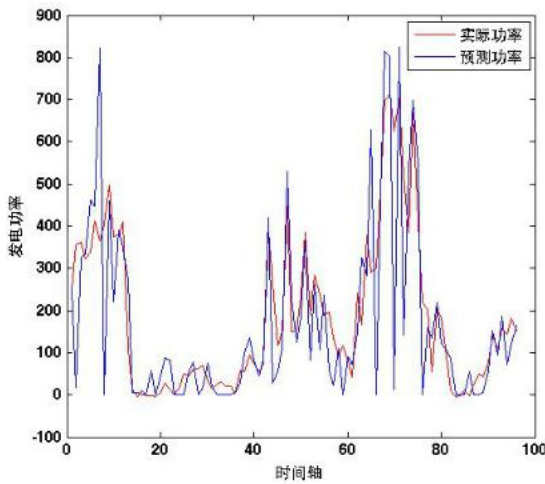


图 (15) a 时段 P_A 功率预测曲线

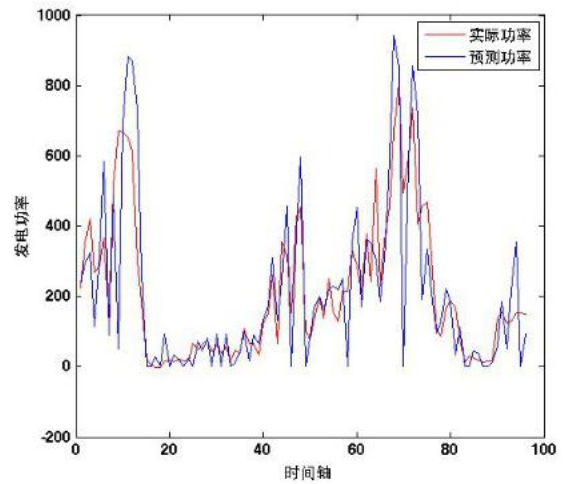


图 (16) a 时段 P_B 功率预测曲线

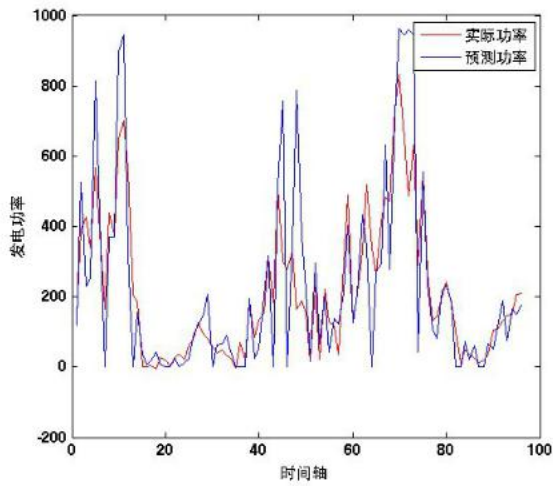


图 (17) a 时段 PC 功率预测曲线

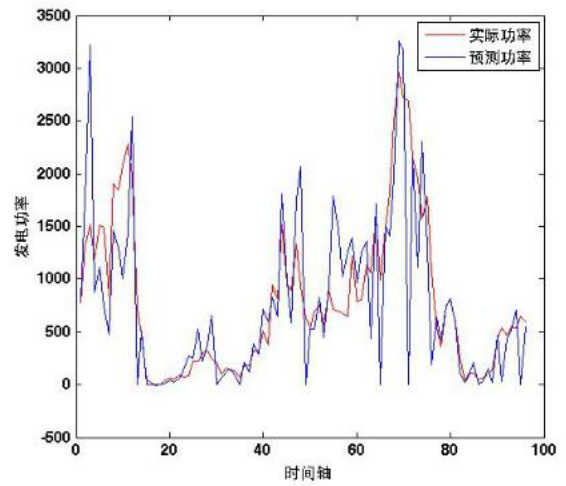


图 (18) a 时段 P4 功率预测曲线

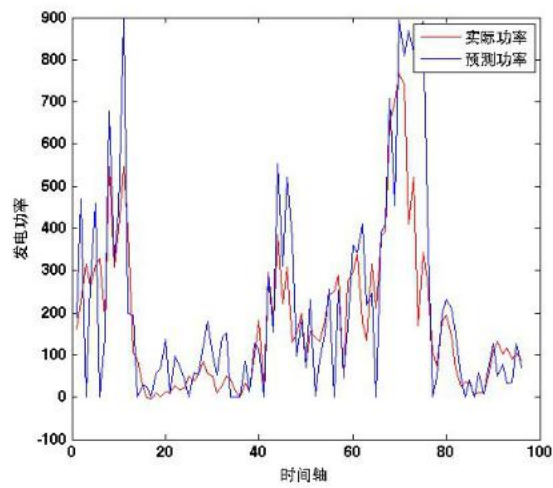


图 (19) a 时段 PD 功率预测曲线

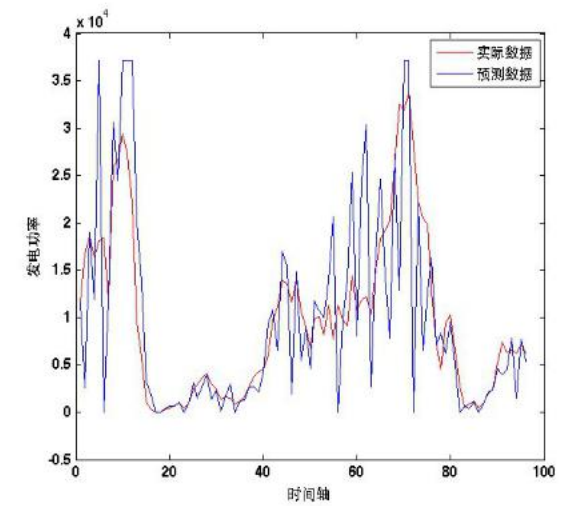


图 (20) a 时段 P58 功率预测曲线

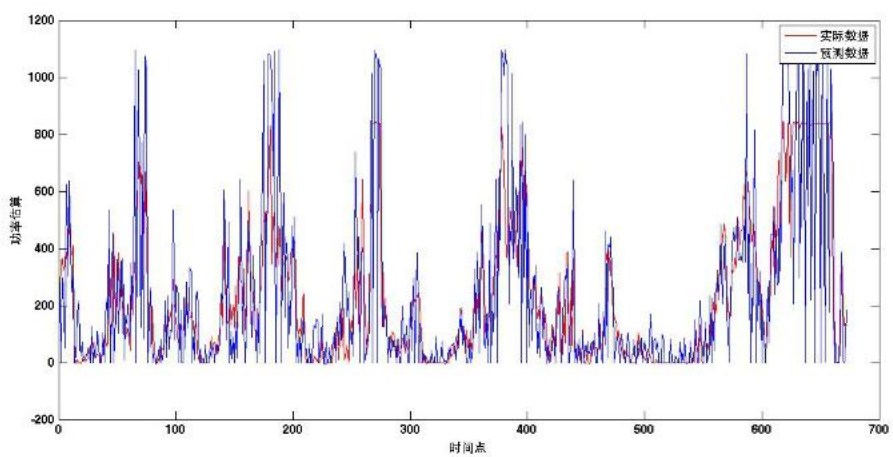
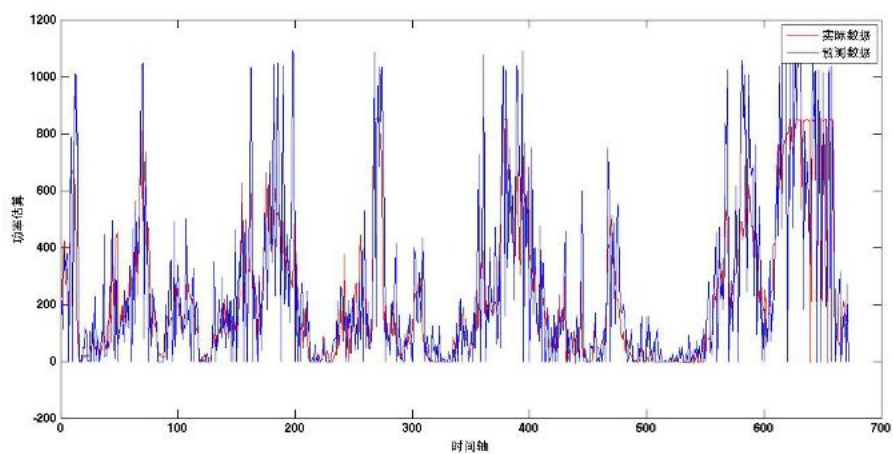
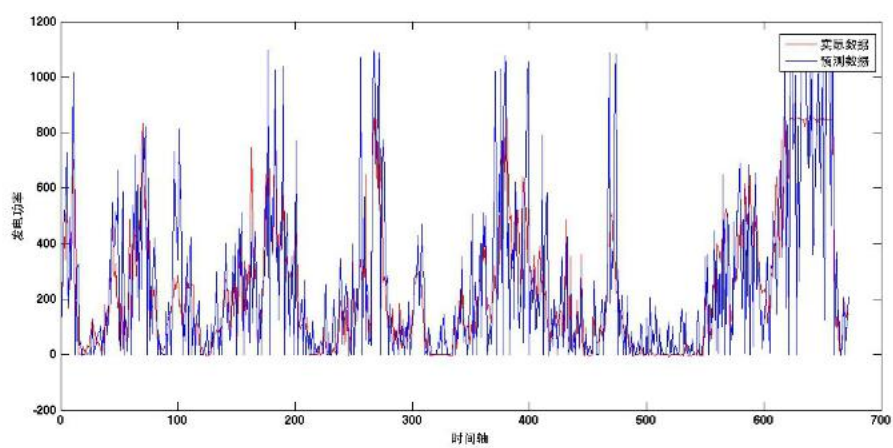


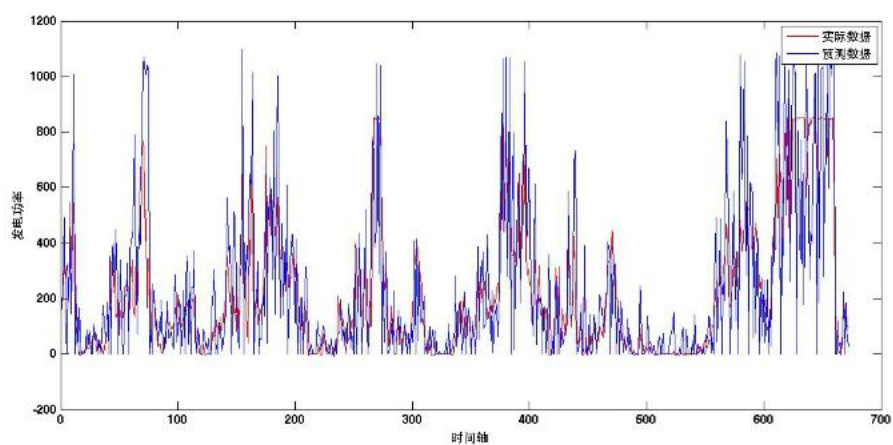
图 (21) b 时段 PA 功率预测曲线



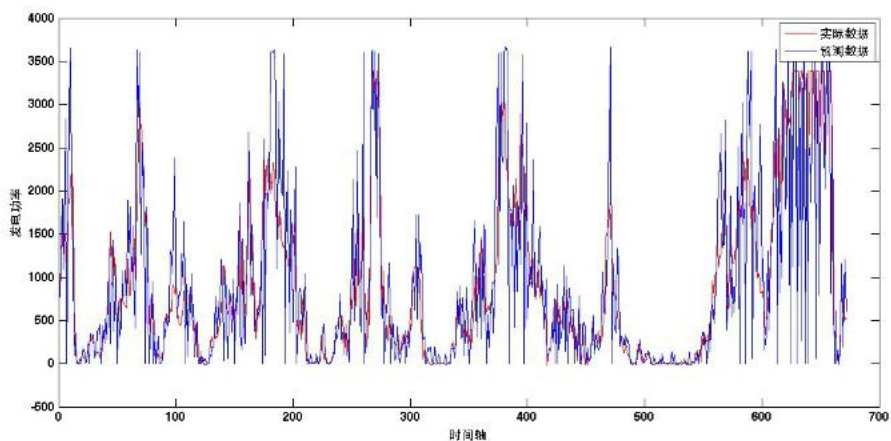
图（22）b 时段 PB 功率预测曲线



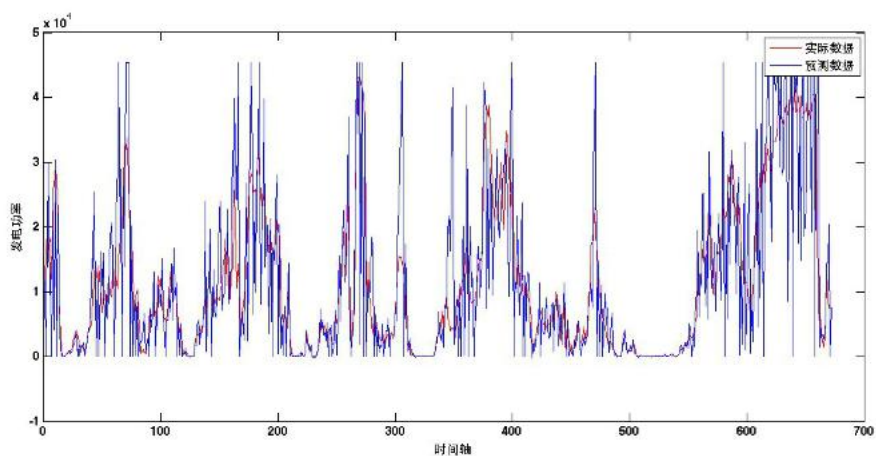
图（23）b 时段 PC 功率预测曲线



图（24）b 时段 PD 功率预测曲线



图（25）b 时段 P4 功率预测曲线



图（26）b 时段 P58 功率预测曲线

卡尔曼滤波模型得到的各项评价指标结果如表 5，表 6。

	P_A	P_B	P_C	P_D	P_4	P_{58}
均方百分比误差/%	8.81	5.75	6.16	7.45	4.79	6.84
平均相对误差/%	56.73	46.65	50.29	59.4	38.34	34.44
精确度/%	86.1	85.59	84.51	84.23	81.37	95.03
合格率/%	92.71	93.75	93.75	90.62	79.17	98.96

表 5 卡尔曼滤波模型 a 时段各项指标结果

	P_A	P_B	P_C	P_D	P_4	P_{58}
均方百分比误差/%	6.74	6.16	6.24	6.77	5.73	6.46
平均相对误差/%	48.95	56.95	49.71	48.49	39.07	49.83
平均精确度/%	87.48	89.59	86.21	86.48	90.16	93.23

平均合格率/%	94.71	93.75	94.79	93.72	96.88	97.92
---------	-------	-------	-------	-------	-------	-------

表 6 卡尔曼滤波模型 b 时段各项指标结果

通过分析该模型得到的功率预测曲线图及相关数据,我们发现该模型的预测结果曲线和实际功率曲线在形状上比 ARMA 预测结果更为接近,在风电功率变化规律改变时,卡尔曼滤波有时能捕捉到变化信息,模型给出的预测值 y_t 滞后于实际值的概率相对降低,使“时滞”问题较为缓和,所以,应用卡尔曼滤波法预测风电功率不仅提高了预测精度,而且在一定程度上解决了时间序列分析法的预测时延问题。但由于卡尔曼滤波方法受到 ARMA 方程的限制,使得在预测效果上仍有上升的空间。

1.3. 小波神经网络预测模型

上文的 ARMA 模型和卡尔曼滤波模型都属于线性模型,都必须先对模型结构做出假设,然后对模型参数的估计得到预测值。因此,模型结构的合理与否,直接影响到最终预测的精度。由于风电场功率具有高度的不确定性,因而单一的线性预测模型不足以挖掘风电功率数据中的所有信息。而神经网络具有自学习、自组织和自适应性,可以充分逼近任意复杂的非线性关系,所以本文选择小波神经网络方法对风电功率进行非线性预测研究。

1.3.1. 小波神经网络法基本原理

小波神经网络是一种以 B P 神经网络拓扑结构为基础,把小波基函数作为隐含层节点的传递函数,信号前向传播的同时误差反向传播的神经网络。小波神经网络的拓扑结构如图 (27)。

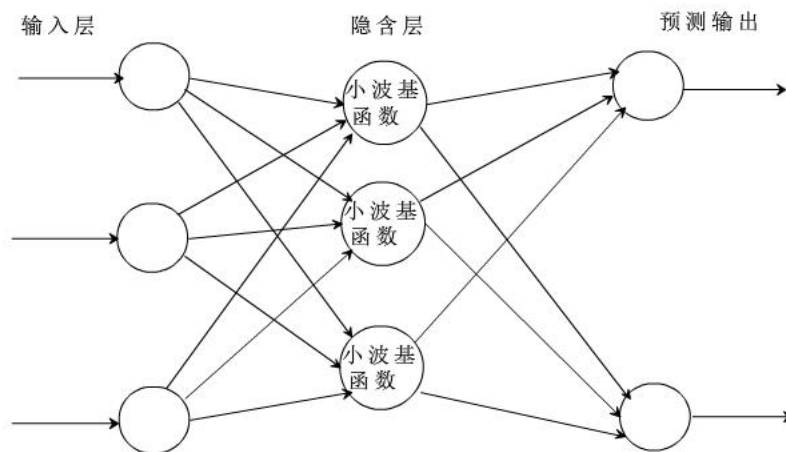


图 (27) 小神经网络拓扑结构

图 (15) 中, $X_1, X_2, \dots, X_k$ 是小波神经网络的输入参数, $Y_1, Y_2, \dots, Y_m$ 是小波神经网络的预测输出, ω_{ij} 和 ω_{jk} 为小波神经网络权值。

在输入信号序列为 $x_i (i=1, 2, \dots, k)$ 时, 隐含层输出计算公式为

$$h(j) = h_j \left[\frac{\sum_{i=1}^k \omega_{ij} x_i - b_j}{a_j} \right] \quad j=1, 2, \dots, l \quad (22)$$

其中， $h(j)$ 为隐含层第 j 个节点输出值； ω_{ij} 为输入层和隐含层的连续权值； b_j 为小波基函数的平移因子； a_j 为小波基函数 h_j 的伸缩因子； h_j 为小波基函数。

该模型中采用的小波基函数为 Morlet 母小波基函数，数学公式为：

$$y = \cos(1.75x)e^{-x^2/2} \quad (23)$$

小波神经网络输出层计算公式为：

$$y(k) = \sum_{i=1}^l \omega_{ik} h(i) \quad k=1, 2, \dots, m \quad (24)$$

其中， ω_{ik} 为隐含层到输出层权值； $h(i)$ 为第 i 个隐含层节点的输出； l 为隐含层节点数； m 为输出层节点数。

小波神经网络权值参数修正算法采用梯度修正法修正网络的权值和小波基函数参数，从而使小波神经网络预测输出不断逼近期望输出。小波神经网络修正过程如下。

(1) 计算网络预测误差

$$e = \sum_{k=1}^m yn(k) - y(k) \quad (25)$$

其中， $yn(k)$ 为期望输出； $y(k)$ 为小波神经网络预测输出。

(2) 根据预测误差 e 修正小波神经网络权值和小波基函数系数

$$\omega_{n,k}^{(i+1)} = \omega_{n,k}^i + \omega_{n,k}^{(i+1)} \quad (26)$$

$$a_k^{(i+1)} = a_k^i + a_k^{(i+1)} \quad (27)$$

$$b_k^{(i+1)} = b_k^i + b_k^{(i+1)} \quad (28)$$

其中， $\omega_{n,k}^{(i+1)}$ 、 $a_k^{(i+1)}$ 、 $b_k^{(i+1)}$ 是根据网络预测误差计算得到：

$$\omega_{n,k}^{(i+1)} = -\eta \frac{\partial e}{\partial \omega_{n,k}^{(i)}} \quad (29)$$

$$a_k^{(i+1)} = -\eta \frac{\partial e}{\partial a_k^{(i)}} \quad (30)$$

$$b_k^{(i+1)} = -\eta \frac{\partial e}{\partial b_k^{(i)}} \quad (31)$$

其中， η 为学习效率。

小波神经网络算法训练步骤如下。

步骤 1：网络初始化。随机初始化小波函数伸缩因子 a_k 、平移因子 b_k 以及网络连接权重 ω_{ij} 、 ω_{jk} ，设置网络学习速率 η 。

步骤 2：样本分类。把样本分为训练样本和测试样本，训练样本用于训练网络，测试样本用于测试网络预测精度。

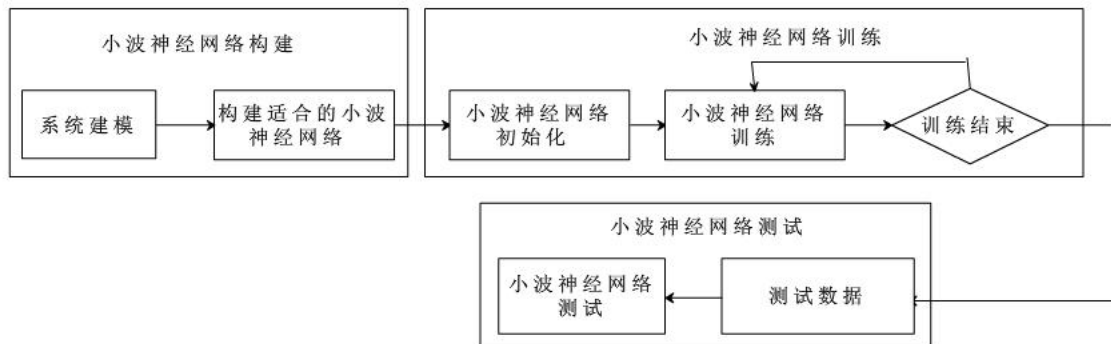
步骤 3：预测输出。把训练样本输入网络，计算网络预测输出并计算网络输出和期望输出地误差 e 。

步骤 4：权值修正。根据误差 e 修正网络权值和小波函数参数，使网络预测值逼近期望值。

步骤 5：判断算法是否结束，如没有结束，返回步骤 3。

1.3.2. 模型建立

首先采集 28 天的风电功率数据，每隔 15min 记录一个时间点，共有 2688 个时间节点的数据，用前 22 天的风电功率数据训练小波神经网络，最后用训练好多的神经网络预测之后的功率数据。基于小波神经网络的风电功率预测算法流程图如图（28）所示。



图（28）小波神经网络测试

小波神经网络的拓扑结构如下图所示：

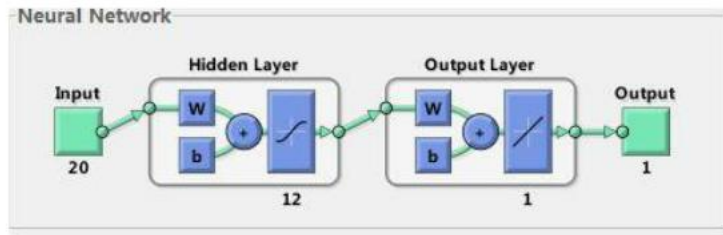


图 (29)

小波神经网络训练：用训练数据训练小波神经网络，网络反复训练 100 次。

神经网络网络测试：用训练好的小波神经网络预测风电功率，并对预测结果进行分析。

1. 3. 3. 预测结果

利用 Matlab 处理数据并进行计算，我们得到基于小波神经网络的风电功率预测结果。

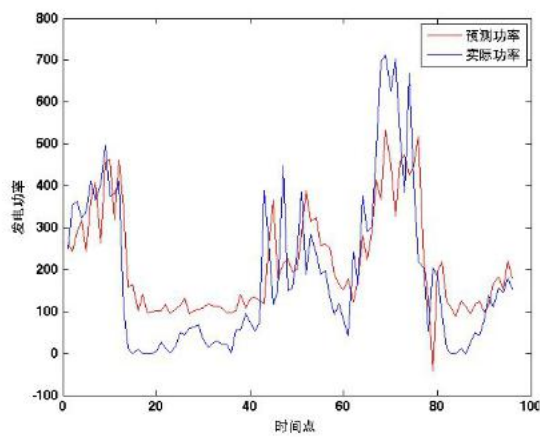


图 (30) a 时段 PA 功率预测曲线

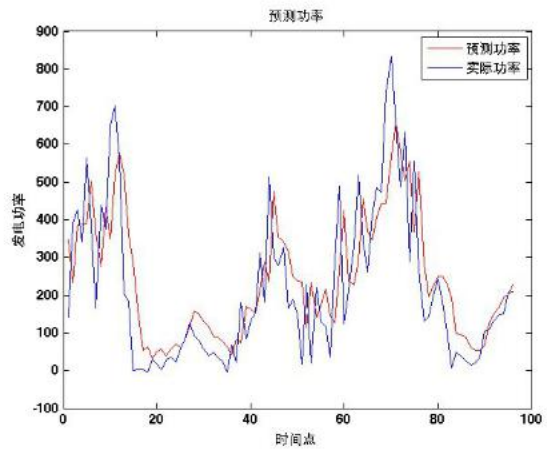


图 (31) a 时段 PC 功率预测曲线

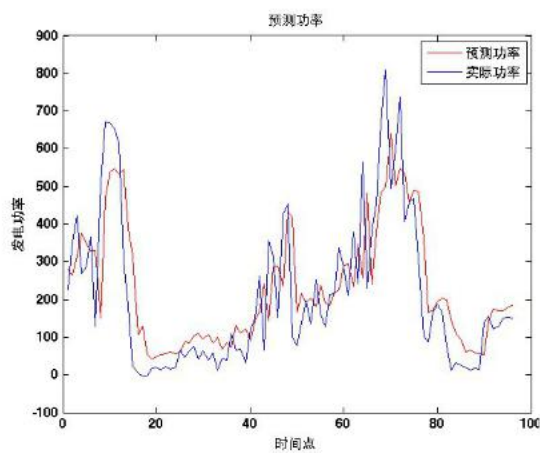


图 (32) a 时段 PB 功率预测曲线

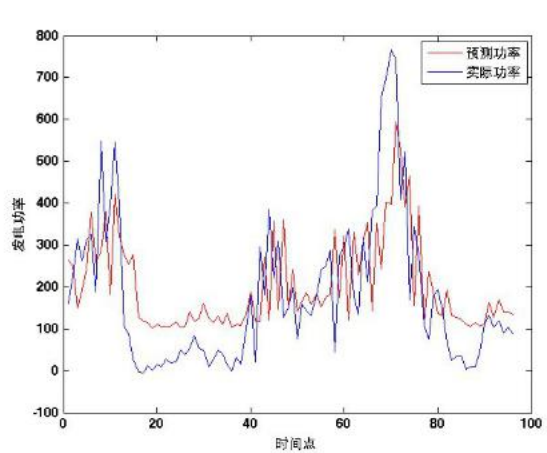


图 (33) a 时段 PD 功率预测曲线

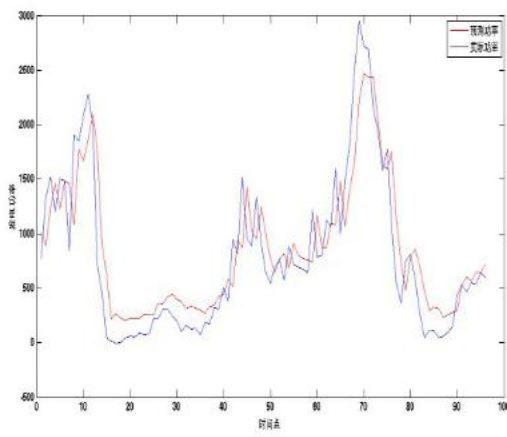


图 (34) a 时段 P4 功率预测曲线

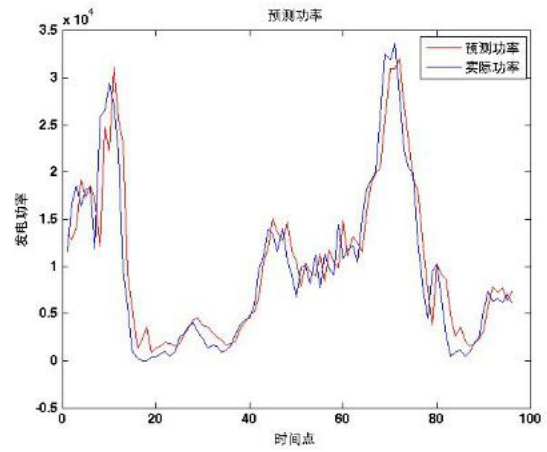


图 (35) a 时段 P58 功率预测曲线

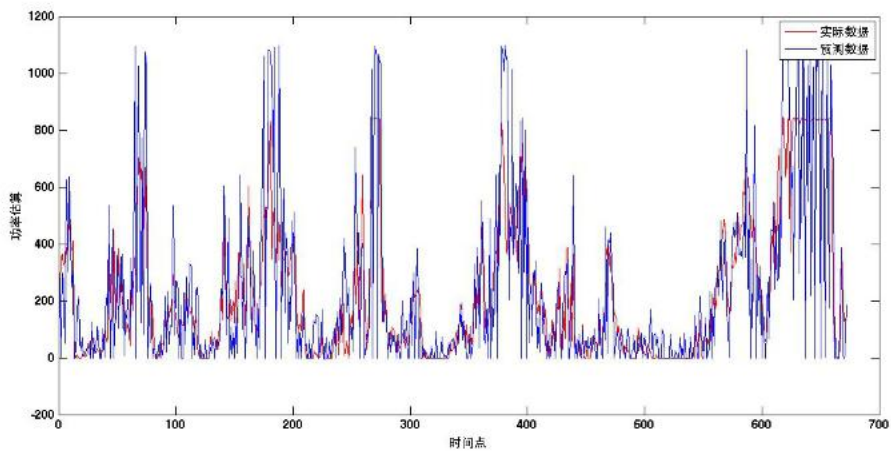


图 (36) b 时段 PA 功率预测曲线

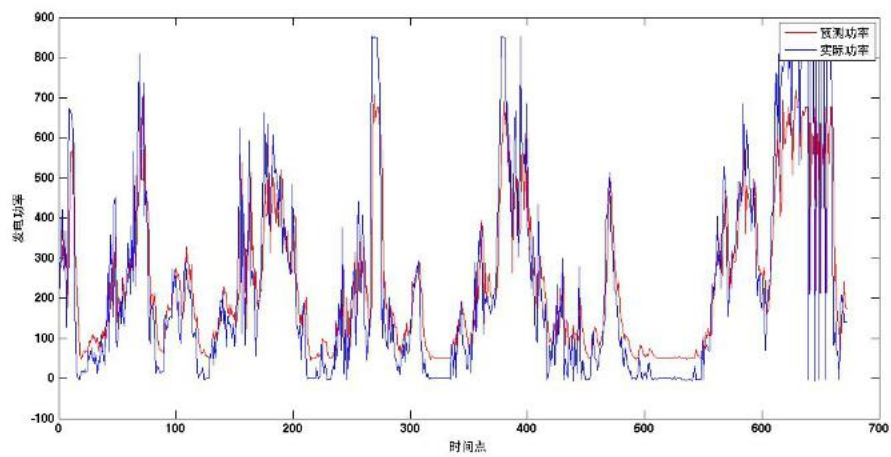
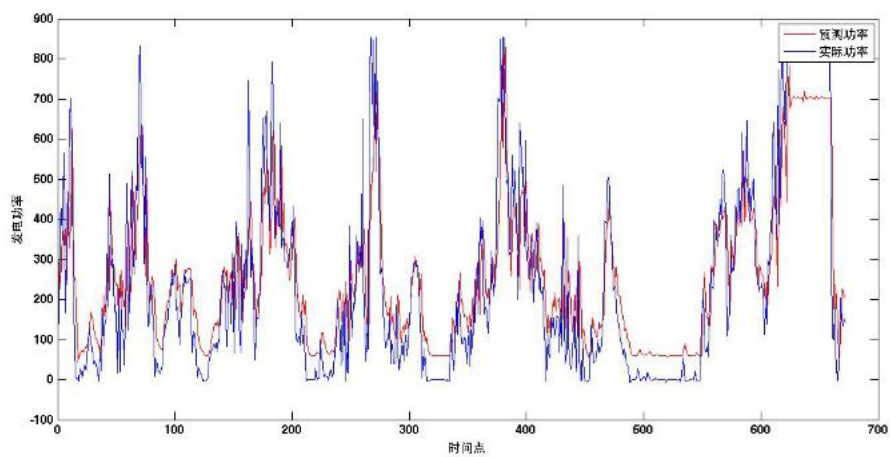
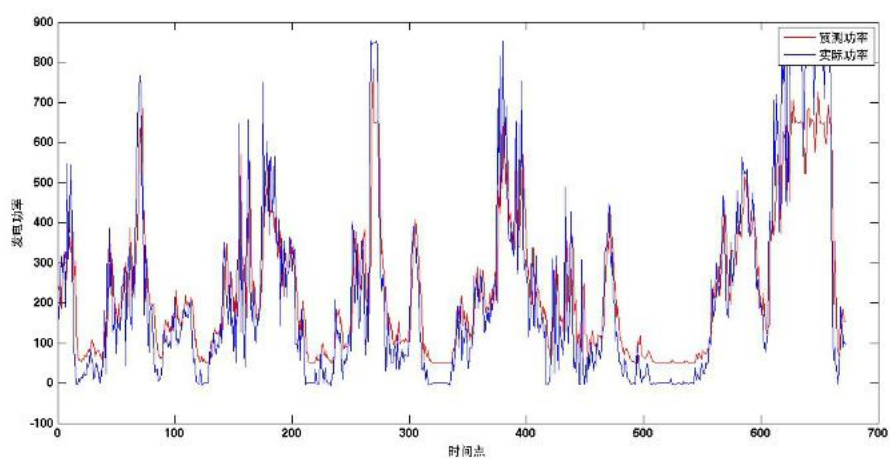


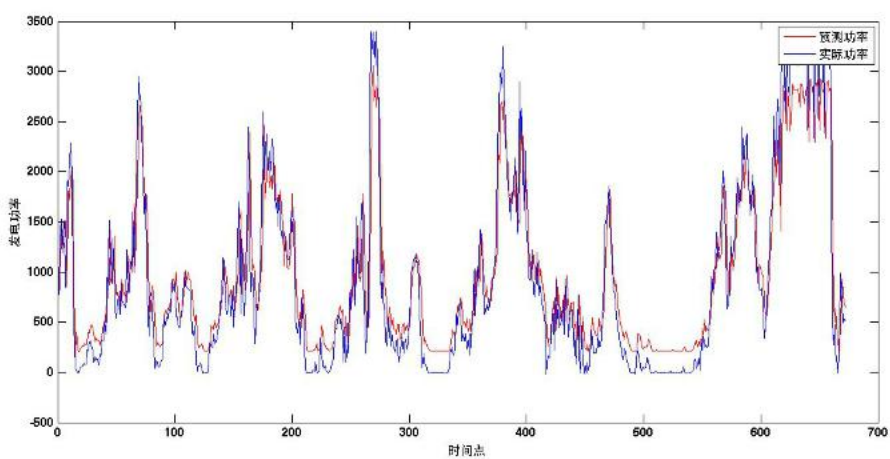
图 (37) b 时段 PB 功率预测曲线



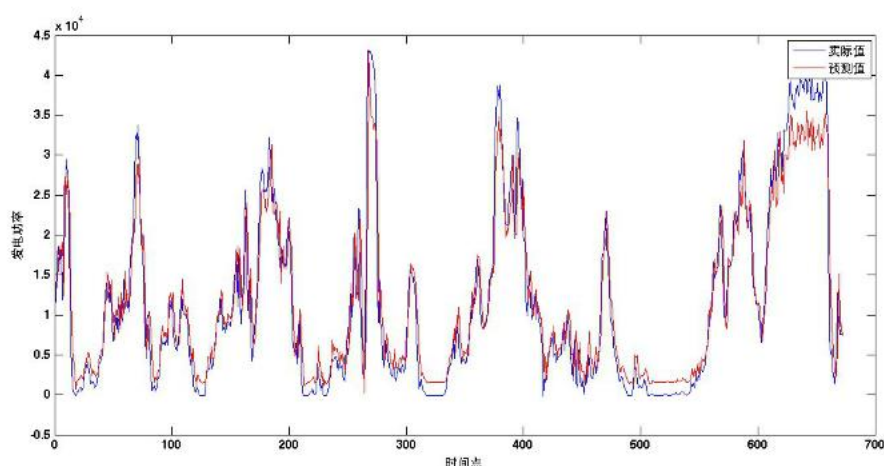
图（38）b 时段 PC 功率预测曲线



图（39）b 时段 PD 功率预测曲线



图（40）b 时段 P4 功率预测曲线



图（41）b 时段 P58 功率预测曲线

小波神经网络模型得到的各项评价指标结果如表 7，表 8。

	P _A	P _B	P _C	P _D	P ₄	P ₅₈
均方百分比误差/%	7.89	11.46	382.53	28.73	6.95	5.05
平均相对误差/%	51.01	61.86	454.95	108.79	27.46	21.63
精确度/%	83.18	83.69	89.86	84.49	92.12	94.73
合格率/%	94.79	98.96	89.58	91.67	98.96	98.96

表 7 小波神经网络模型 a 时段各项指标结果

	P _A	P _B	P _C	P _D	P ₄	P ₅₈
均方百分比误差/%	24.70	51.22	18.71	35.34	8.45	1.76
平均相对误差/%	176.13	290.51	134.03	207.71	54.57	21.62
平均精确度/%	87.46	88.00	90.83	90.9	94.25	95.38
平均合格率/%	96.13	97.62	98.81	97.77	99.85	98.85

表 8 小波神经网络模型 b 时段各项指标结果

小波神经网络模型中我们得到预测结果：单机组的平均精确度达到 85%，合格率达到 93%，而 58 机组的精确度达到 94%，合格率达到 97%；小波神经网络的预测结果已经相当精确。对小波神经网络预测曲线与线性预测模型的预测曲线进行对比，可以看到：神经网络对于风电功率的描绘更加平缓，但神经网络的预测结果的平均相对误差和卡尔曼滤波法预测结果的平均相对误差近似。

1.4. 预测效果分析

本文采用了 ARMA 模型、卡尔曼滤波预测算法和小波神经网络算法对 a 时段和 b 时段的各机组的风电功率数据样本进行了预测。分析表 1~表 6 的预测效果评价指标，我们得到以下认识：

- (1) 应用上述三种风电功率预测模型进行风电预测都能达到较好的预测效果，预测曲线基本能较好地反映风电功率的变化趋势。比较单台机组与多台机组的风电功率预测误差及精确度等，可以看出多台机组的预测精度更高，这是因为单台机组受风的随机特性的影响较大。
- (2) ARMA 模型是风电功率预测的最常用方法，这是因为 ARMA 模型利用历史风电功率数据建立预测方程，根据风电功率的连续性，通过统计分析，进一步推测未来时刻风电功率发展的趋势，方法比较简单，并且预测效果也比较稳定，适合于工程应用，但是缺点在于 ARMA 模型为线性模型，使得预测精度的提高受到限制。
- (3) 卡尔曼滤波预测模型是一种最小方差估计的递推式滤波方法，能够前一时刻的预测误差及时修正下一时刻的预测值，对预测效果的改善有一定的作用。但是，卡尔曼滤波预测模型是基于 ARMA 模型的，并且不能很好的改善 ARMA 模型预测的滞后特征，使得预测精度仍有提高的空间。
- (4) 小波神经网络是风电功率预测方法中有代表性的一种。由于线性预测模型不足以挖掘风电功率中的所有信息，而神经网络具有自学习、自组织和自适应性，可以充分逼近任意复杂的非线性关系，所以神经网络应用于风电功率预测能够达到较好的预测效果。

综上所述，对于单台机组进行风电功率预测时，采用线性方法如 ARMA 和卡尔曼滤波等，能够满足建模需要，方法基本可用。对精确度要求较高时，应该首先考虑小波神经网络法。在工程应用中，对于风电场的风电功率进行预测时，首先应该分析该序列的特性，从而选择合理有效地预测方法。对于多机组风电功率的预测，小波神经网络能更好的挖掘时间序列的非线性规律，起到良好的预测作用。

2. 风电机组的汇聚对于预测结果误差的影响

我国主要采用集中开发的方式开发风电，各风电机组功率汇聚通过风电场或风电场群接入电网。众多风电机组的汇聚会改变风电功率波动的属性，从而可能影响预测的误差。

基于问题 1 的预测结果我们得到三种预测方法下的单台风电机组功率 (P_A , P_B , P_C , P_D) 的相对预测误差与多机总功率 (P_4 , P_{58}) 预测的相对误差，如表 9。

	P_A	P_B	P_C	P_D	P_4	P_{58}
ARMA 预测模型	57.07	464.1	47.12	165.40	85.14	52.28
卡尔曼滤波预测模型	56.73	46.65	50.29	59.40	38.34	34.44
小波神经网络预测模型	51.01	61.86	451.95	108.79	27.46	21.63

表 9 单机/多机预测功率相对误差比较

由上表可知，对于每种风电功率预测模型的预测结果，多机总功率 P_{58} 或 P_4 预测的相对误差大部分情况下比单机预测的相对误差小。可能原因是，风力在空间中的分布具有波动性和不均等性，单台发电机覆盖的空间范围有限因此随

机波动的程度较大。由多台机组构成的分析系统覆盖范围广，与局部波动性相户弥补使整体稳定性较高，因此多台机组预测比单台机组预测相对误差小。

从以上的比较中我们发现风电机组的汇聚对于预测结果误差有影响。显然，风电是一种间歇性、波动性电源，大规模风电的汇聚对电力系统的安全稳定运行带来了挑战。为克服风电波动对电力系统运行的不利影响，必须对电力系统进行有效的计划和调度，加大电力系统的旋转备用容量。旋转备用容量的增加间接地增加了风力发电的运营整体成本，所以需要风电场的风电功率进行实时预测。因此风电功率预测对于电网安全经济调度、电力市场及风电场运行就显得意义重大。

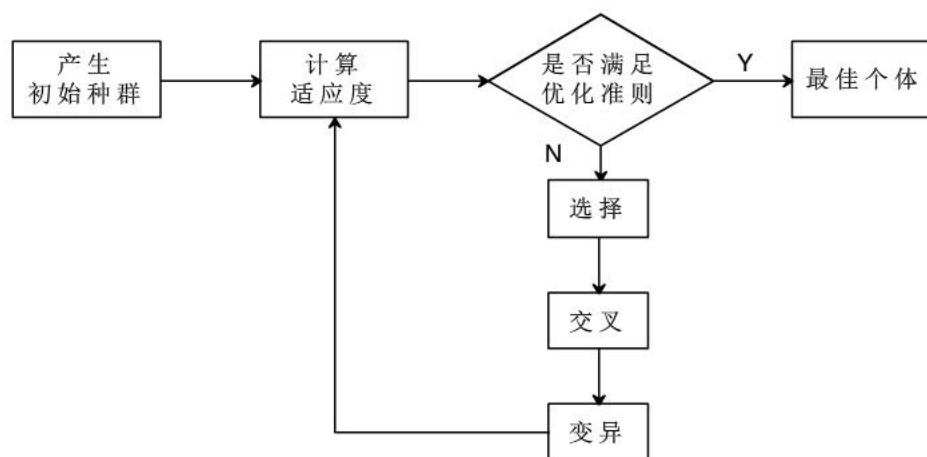
3. 风电功率实时预测的改进

在对风电功率实时预测的研究中，我们发现神经网络模型具有较好的预测精度，同时也发现小波神经网络模型存在一些问题，影响预测的精度。我们发现小波神经网络对与峰值数据的预测精度不是非常理想。为了进一步提高预测精度，减小网络对峰值的预测误差，我们采用遗传算法先优化网络的拓补结构，再优化网络参数。最后，我们用改进的模型进行了仿真，取得了比较理想的预测结果。

3.1 遗传神经网络算法基本原理

3.1.1 遗传算法

遗传算法（Genetic Algorithm, GA）是一种基于自然选择和基因遗传学原理的优化搜索方法。它将“优胜劣汰，适者生存”的生物进化原理引入待优化参数形成的编码串群体中，按照一定的适配值函数及一系列的遗传操作对个体进行了筛选，从而使适配值高的个体被保留下来，组成新的群体，新群体中包含上一代的大量信息，并且引入新的优于上一代的个体。这样的周而复始，群体中的适应度不断提高，直到满足一定的条件为止。遗传算法的基本原理如下图所示：



图（42）

遗传算法的特点如下：

- 1) 遗传算法对参数的编码进行操作，而非对参数本身。
- 2) 遗传算法从许多点开始进行操作，并非局限于一点，因而可以有效地防止搜索过程中收敛于局部最优解。
- 3) 遗传算法通过目标函数来计算适配值，而不需要其它推导和附加信

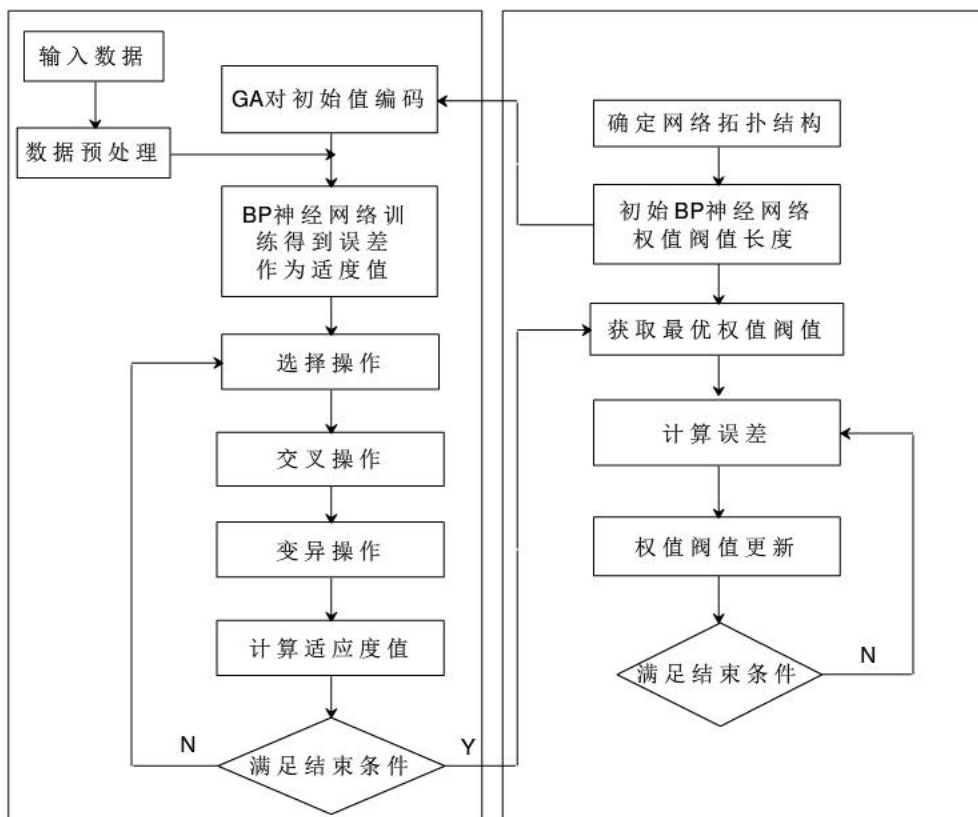
息，对问题的依赖小

- 4) 遗传算法的寻优规则是由概率决定的，而非确定性的。
- 5) 遗传算法在解空间进行高效启发式的搜索，而非穷举或完全随机搜索。
- 6) 遗传算法对于待寻优的函数基本无限制，它既不要求连续，也不要求可微，既可以是数学表达式的显函数，又可以是映射矩阵甚至是神经网络等隐函数，因而应用范围较广。
- 7) 遗传算法具有并行运算的特点，因而可以通过大规模并行运算来提高计算速度。
- 8) 遗传算法更适合大规模复杂问题的优化。
- 9) 遗传算法计算简单，功能强。

遗传算法提供了一种求解复杂系统优化问题的通用框架。

3.1.2 遗传算法优化神经网络

遗传算法与神经网络算法的结合就是利用 GA 优化网络的拓补结构，如网络层数和每层的节点数，以及各层节点间的连接关系。根据某些性能评价准则（如学习速度，泛化能力或结构复杂程度等）搜索结构空间中满足问题要求的最佳神经网络结构。遗传神经网络算法流程如下图所示：



图（43）

3.2 遗传神经网络模型的建立

我们根据遗传算法和 BP 神经网络理论，在 MATLAB 软件中编程实现了基于遗

传算法优化 BP 神经网络非线性系统拟合算法。

我们首先设置了种群的规模。根据输入数据的维数，我们设定种群的规模为 10，进化次数为 50 次，交叉概率为 0.4，变异概率为 0.2。

3.2.1 适应度函数

我们根据个体得到了 BP 神经网络的初始权值和阈值，用训练数据训练 BP 神经网络后预测系统输出，把预测输出和期望输出之间的绝对值和 E 作为个体适应度值 F ，计算公式如下：

$$F = k \left(\sum_{i=1}^n \text{abs}(y_i - o_i) \right) \quad (32)$$

式中， n 为网络输出节点数； y_i 为 BP 神经网络第 i 个节点的期望输出； o_i 为第 i 个节点的预测输出； k 为系数。

3.2.2 选择操作

我们接着定义了遗传算法的选择操作。我们选择轮盘赌法，即基于适应度比例的选择策略，每个个体 i 的选择概率 p_i 为

$$f_i = k / F_i \quad (33)$$

$$p_i = \frac{f_i}{\sum_{j=1}^N f_j} \quad (34)$$

式中， F_i 为个体 i 的适应度值；由于适应度值越小越好，所以在个体选择之前对适应度值求倒数； k 为系数； N 为种群个体数目。

3.2.3 交叉操作

我们接着定义了交叉操作。个体我们采用实数编码，所以交叉操作方法采用实数交叉法，第 k 个染色体 a_k 和第 l 个染色体 a_l 在 j 位的交叉操作方法如下：

$$a_{kj} = a_{kj}(1-b) + a_{lj}b \quad (35)$$

$$a_{lj} = a_{lj}(1-b) + a_{kj}b \quad (36)$$

式中， b 是 $[0, 1]$ 区间的随机数。

3.2.4 变异操作

选取第 i 个个体的第 j 个基因 a_{ij} 进行变异，变异的操作方法如下：

$$a_{ij} = \begin{cases} a_{ij} + (a_{ij} - a_{\max}) * f(g) & r \geq 0.5 \\ a_{ij} + (a_{\min} - a_{ij}) * f(g) & r < 0.5 \end{cases} \quad (37)$$

其中, $a_{\max}$ 为基因 a_{ij} 的上界; $a_{\min}$ 为基因 a_{ij} 的下界; $f(g) = r_2(1 - g / G_{\max})$; r_2 为一个随机数; g 为当前迭代次数; $G_{\max}$ 是最大进化次数; r 为 $[0, 1]$ 间的随机数

3.2.5 主函数

程序段代码如下

```
%去训练数据和预测数据
input_train=input';
output_train=output';
input_test=input_test';
%数据归一化
[inputn,inputps]=mapminmax(input_train);
[outputn,outputps]=mapminmax(output_train);
%构建网络
net=newff(inputn,outputn,8);
%BP神经网络参数
net.trainParam.epochs=30;
net.trainParam.lr=0.01;
net.trainParam.goal=0.0000004;
%BP神经网络训练
net=train(net,inputn,outputn);
%% BP网络预测
%预测输入数据归一化
inputn_test=mapminmax('apply',input_test,inputps);
%网络预测输出
an=sim(net,inputn_test);
%网络输出反归一化
BPoutput=mapminmax('reverse',an,outputps);
%% 结果分析
figure(1)
```

```

plot(BPoutput,':og')

holdon

plot(output_test,'-*');

legend('预测输出','期望输出','fontsize',12)

title('BP网络预测输出','fontsize',12)

[xx,~]=size(BPoutput);

plot(1:xx,BPoutput,1:xx,output_test)

xlabel('样本','fontsize',12)

ylabel('输出','fontsize',12)

print-dtiff-r6004-3

%预测误差

error=BPoutput-output_test;

figure(2)

plot(error,'-*')

title('神经网络预测误差')

figure(3)

plot((output_test-BPoutput)./BPoutput,'-*');

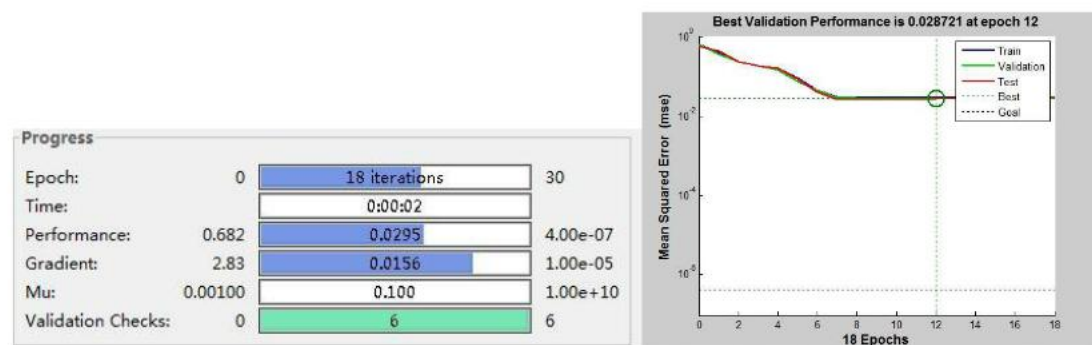
title('神经网络预测误差百分比')

errorsum=sum(abs(error))

```

3.3 模型检验

我们利用建立的遗传神经网络模型分别对 a 时间段 A 号风力发电机功率、 a 时间段 58 台风力发电机总功率、 b 时间段 B 号风力发电机功率进行仿真，网络的训练结果如下图所示：



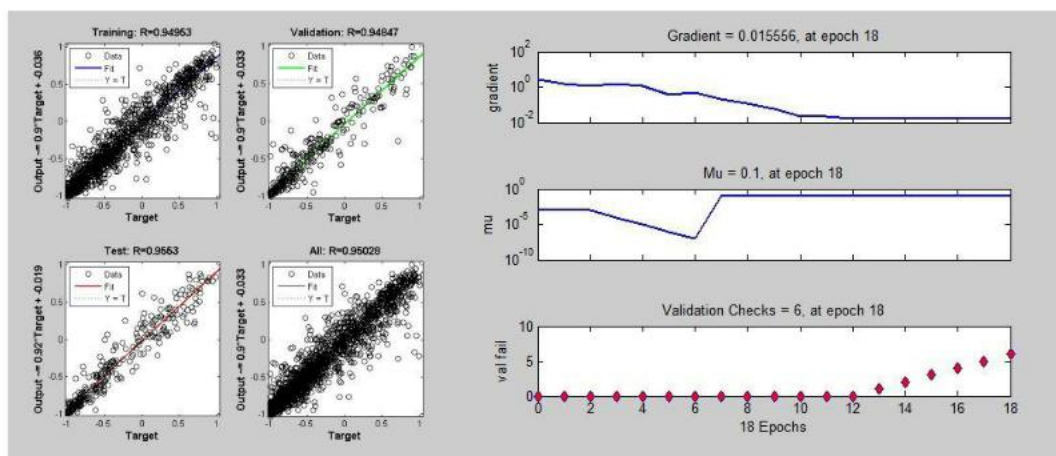


图 (44)

b 时间段 58 台风力发电机总功率进行仿真结果如下：

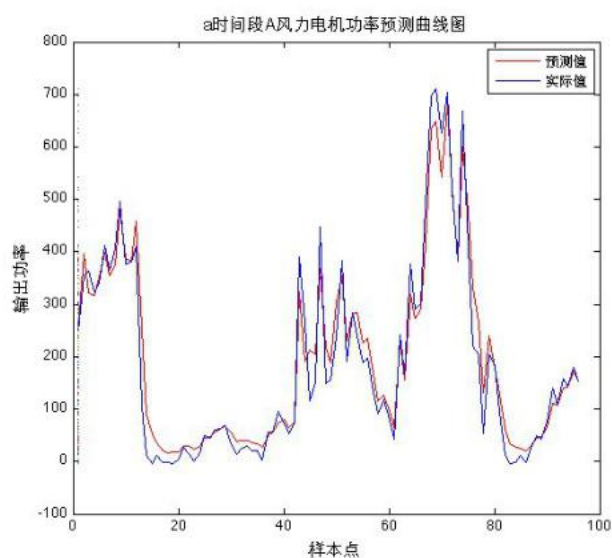
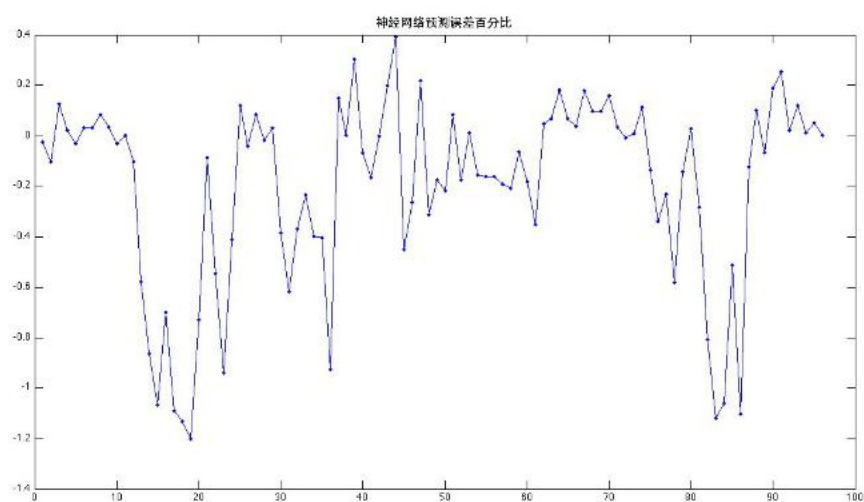
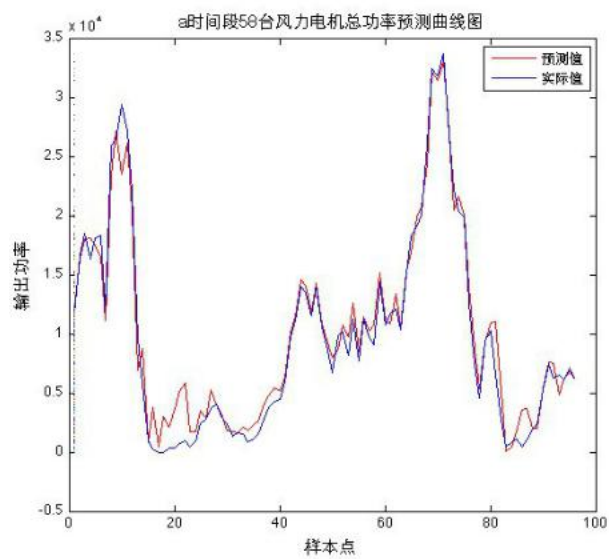


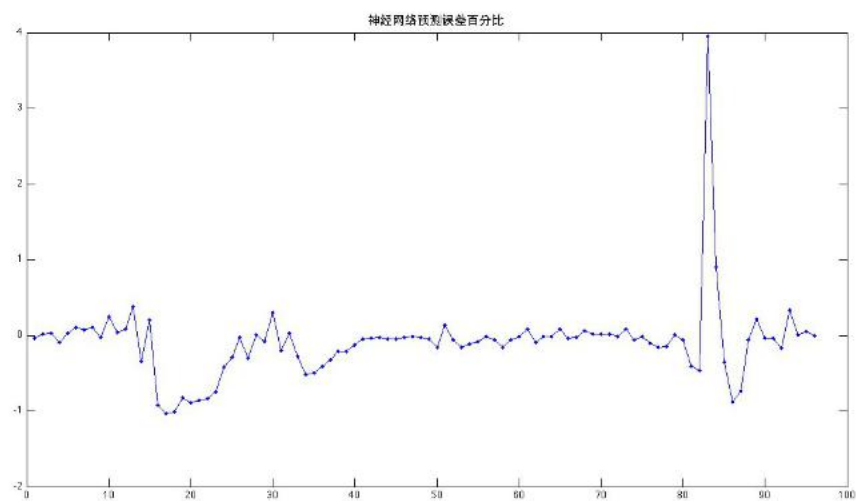
图 (45) a 时段 A 发电机功率预测曲线



图（46）a 时段 A 发电机功率预测误差百分比曲线



图（47）a 时段 58 台发电机总功率预测曲线



图（48）a 时段 58 台发电机总功率预测误差百分比曲线

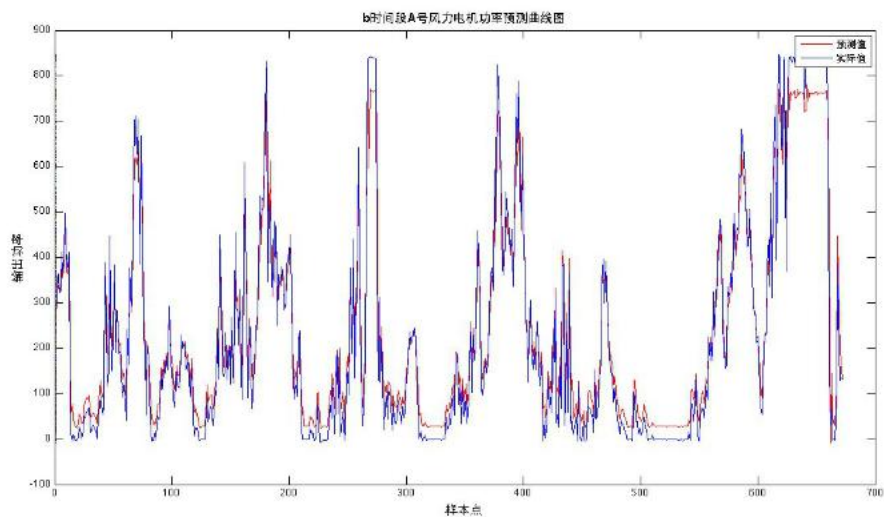


图 (49) b 时段 A 发电机功率预测曲线图

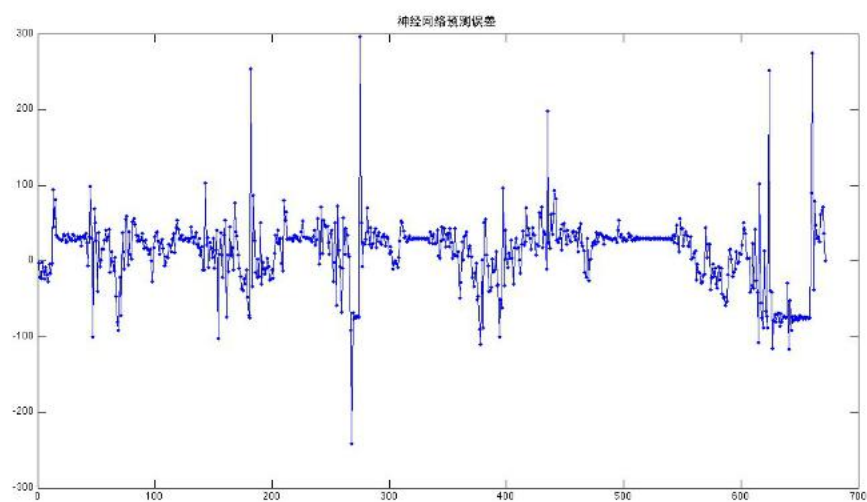
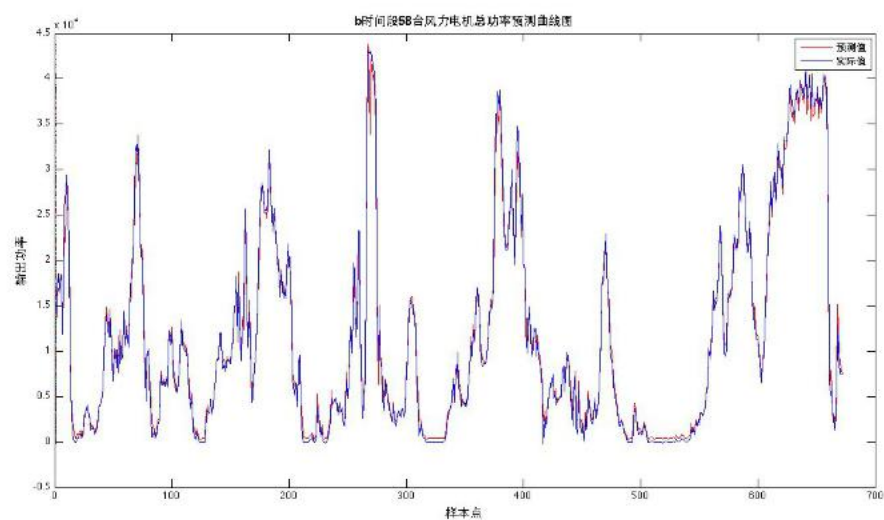
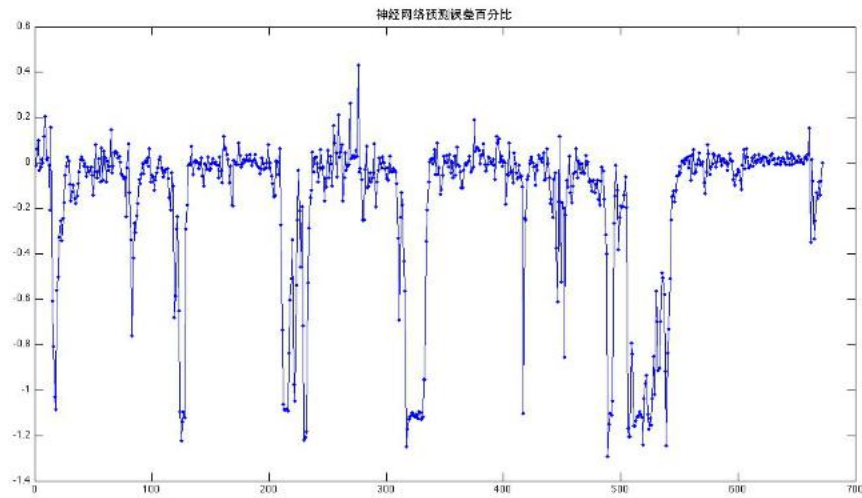


图 (50) a 时段 A 发电机功率预测误差百分比曲线



图（51）b时段 58 台发电机总功率预测曲线图



图（52）a时段 58 台发电机总功率预测误差百分比曲线

我们发现模型对于峰值有着很理想的预测精度，模型整体的预测精度得到了很大的提高。并且，模型对于发电机发电功率随时间的变化趋势预测非常准确，网络的泛化能力很好，时间延迟现象得到了很大的改善。

我们对仿真结果进行分析我们发现在大多数观测点误差百分比在零附近上下波动。误差百分比比较大的点会随机出现，没有确定规律。我们认为风速与风向不仅仅受当地地理位置等确定性因素的影响，还受人类活动、大气异常运动等突加因素的影响。因而，风速与风向随时间的变化关系不是确定的，受突加扰动的影响。受扰动影响的风速与风向决定了发电机组的功率的变化规律具有一定的不确定性。由于神经网络模型只能学习并刻画已有的规律，因而产生了一定的误差。我们认为突加的扰动是阻碍风电功率实时预测精度进一步改善的主要因素。在现实环境下，突加的扰动无法彻底消除，因而风电功率预测精度不可能无限提高。

六、结论和展望

1. 结论

本文利用 ARMA 模型、卡尔曼滤波预测模型、小波神经网络预测模型对风电场的风电功率进行了预测，论证了上述方法的可行性，并对这些算法的精确度进行了比较及探讨。在这些模型的基础上，我们改进了小波神经网络算法，又提出了遗传神经网络预测模型，预测结果表明，该模型的精确度更高。

2. 有待解决的问题

（1）虽然风电功率时间序列已经包含了各种与之相关的影响因素（天气、大气情况、地形因素等）的信息。但由于原始数据的限制，没能在风电功率中引入天气、温度、风向等其他因素的影响。如果能引入多种影响的变量，丰富样本空间中影响因素的数据，缩短预测的时间，将能提高预测精度。

（2）本着预测精度和建模效率统一的原则，论文中对于隐层节点数、隐层

和输出层传递函数的选择都是通过手动调节的，如果进一步研究的话，应该可以实现计算机自动调节。

（3）随着科技的发展，天气预报将能提供较为准确的分时段气象数据，如果能够在预测时考虑到预测点的天气状态特别是风速预报，将是一件非常有意义的事。

（4）小波神经网络在训练速度、预测结果稳定性和预测精度上仍有提升的空间。

七、参考文献

- 【1】Bailey B, Brower MC, Zack J. Short-term wind forecasting[C]. European Wind Energy Conference, Nice France, 1999:1062-1065
- 【2】www.cwpc.cn/cwpc/zh-cn/system
- 【3】M.Alexiadis, P.Dokopoulos, H.Sahsamanoglou, I.Manousaridos.Short term forecasting of wind speed and related electrical power. Solar Energy.1998,63(1)
- 【4】邓自立, 卡尔曼滤波与维纳滤波[M], 哈尔滨: 哈尔滨工业大学出版社, 2001
- 【5】丁明, 张立军, 吴义纯, 基于时间序列分析的风电场风速预测模型[J], 电力自动化设备, 2008, 8 (25): 32, 34
- 【6】时庆华, 高山, 基于 ARMA 和神经网络的风电场短期风速预测, 2009
- 【7】时庆华, 高山, 基于 ARMA 和卡尔曼滤波的风电场风电功率预测研究, 第 25 届全国高校电力系统及其自动化专业学术年会, 2009
- 【8】蒋宗礼, 人工神经网络导论[M], 北京: 高等教育出版社, 2001:97-106
- 【9】魏晓霞, 我国风电发展存在的问题和应对措施[J], 电力技术经济, 2009, 21 (6): 23-26
- 【10】韩爽, 风电场功率预测方法研究[D], 华北电力大学, 2008
- 【11】Pinson P, Kariniotakis G N. Wind Power Forecasting Using Fuzzy Neural Networks Enhanced with On-line Prediction Risk Assessment[A]. in: Power Tech Conference Proceedings 2003 IEEE[C]. Bologna: 2003.
- 【12】Damousis I G, Dokopoulos P. A Fuzzy Expert System for the Forecasting of Wind Speed and Power Generation in Wind Farms[J]. Power Industry Computer Applications, 2001, (4): 63-69.
- 【13】Wang P , Billinton R . Time-sequential Simulation Technique for Rural Distribution System Reliability Cost/worth Evaluation Including Wind Generation as Alternative Supply[J]. IEEE Proceedings on Gener, Transm, and Distrib, 2001, 148(4): 355-360.
- 【14】Li S. Wind Power Prediction Using Recurrent Multilayer Perceptron Neural Networks[A]. in: Power Engineering Society General Meeting, IEEE[C]. 2003. 2325-2330.
- 【15】CAO Lei, LI Ran. Short-Term Wind Speed Forecasting Model for Wind Farm Based on Wavelet Decomposition[J]. IEEE Trans on DRPT, 2008: 2525-2529
- 【16】LI Shu-hui, Wunsch D, O'Hair E, et al. Using Neural Networks to Estimate Wind Turbine Power Generation[A]. in: Power Engineering Society Winter Meeting[C]. 2001. 977.

【17】楼顺天,施阳, 基于 Matlab 的系统分析与设计——神经网络[M]. 西安: 西安电子科技大学出版社, 2000.

附录：

%遗传神经网络主函数

```
input_train=input';
output_train=output';
input_test=input_test';

[inputn,inputps]=mapminmax(input_train);
[outputn,outputps]=mapminmax(output_train);

net=newff(inputn,outputn,12);

net.trainParam.epochs=30;
net.trainParam.lr=0.001;
net.trainParam.goal=0.0000004;

net=train(net,inputn,outputn);

inputn_test=mapminmax('apply',input_test,inputps);

an=sim(net,inputn_test);

BPoutput=mapminmax('reverse',an,outputps);

[~,in]=size(BPoutput);
BPoutput(1:in-1)=BPoutput(2:in);
BPoutput(in)=output_test(in);
figure(1)
plot(BPoutput,'-og')
holdon
plot(output_test,'-*');
```

```

legend('预测值','实际值','fontsize',12)
title('·风力电机功率预测曲线图','fontsize',12)
[xx,~]=size(BPoutput);
plot(1:xx,BPoutput,1:xx,output_test)
xlabel('样本点','fontsize',12)
ylabel('输出功率','fontsize',12)
print-dtiff-r6004-3
%Ô²âÖ²î
error=BPoutput-output_test;

figure(2)
plot(error,'-*')
title('神经网络预测误差')

figure(3)
plot((output_test-BPoutput)./BPoutput,'-*');
title('神经网络预测误差百分比')

errorsum=sum(abs(error));

```

%遗传主函数

```

inputnum=4;
hiddennum=8;
outputnum=1;

input_train=input';
input_test=output';

```

```

output_train=input_test;
output_test=output_test';

[inputn,inputps]=mapminmax(input_train);
[outputn,outputps]=mapminmax(output_train);

net=newff(inputn,outputn,hiddennum);

maxgen=10;
sizepop=10;
pcross=[0.3];
pmutation=[0.1];

numsum=inputnum*hiddennum+hiddennum+hiddennum*outputnum+outputnum
;

lenchrom=ones(1,numsum);
bound=[-3*ones(numsum,1) 3*ones(numsum,1)];

-----

individuals=struct('fitness',zeros(1,sizepop), 'chrom',[]);
avgfitness=[];
bestfitness=[];
bestchrom=[];
for i=1:sizepop
individuals.chrom(i,:)=Code(lenchrom,bound);
    x=individuals.chrom(i,:);
    individuals.fitness(i)=fun(x,inputnum,hiddennum,outputnum,net,inputn,outputn);
end

[bestfitnessbestindex]=min(individuals.fitness);

```

```

bestchrom=individuals.chrom(bestindex,:);
avgfitness=sum(individuals.fitness)/sizepop;
trace=[avgfitnessbestfitness];

for i=1:maxgen
    i
    individuals=Select(individuals,sizepop);
    avgfitness=sum(individuals.fitness)/sizepop;

    individuals.chrom=Cross(pcross,lenschrom,individuals.chrom,sizepop,bou
nd);

    individuals.chrom=Mutation(pmutation,lenschrom,individuals.chrom,sizep
op,i,maxgen,bound);

    for j=1:sizepop
        x=individuals.chrom(j,:);

        individuals.fitness(j)=fun(x,inputnum,hiddennum,outputnum,net,inp
utn,outputn);
    end

    [newbestfitness,newbestindex]=min(individuals.fitness);
    [worestfitness,worestindex]=max(individuals.fitness);
    ifbestfitness>newbestfitness
        bestfitness=newbestfitness;
        bestchrom=individuals.chrom(newbestindex,:);
    end
    individuals.chrom(worestindex,:)=bestchrom;
    individuals.fitness(worestindex)=bestfitness;

    avgfitness=sum(individuals.fitness)/sizepop;

    trace=[trace;avgfitnessbestfitness];

```

```
end

figure(1)

[r c]=size(trace);

plot([1:r]',trace(:,2),'b--');

x=bestchrom;

w1=x(1:inputnum*hiddennum);

B1=x(inputnum*hiddennum+1:inputnum*hiddennum+hiddennum);

w2=x(inputnum*hiddennum+hiddennum+1:inputnum*hiddennum+hiddennum+
hiddennum*outputnum);

B2=x(inputnum*hiddennum+hiddennum+hiddennum*outputnum+1:inputnum*
hiddennum+hiddennum+hiddennum*outputnum+outputnum);

net.iw{1,1}=reshape(w1,hiddennum,inputnum);
net.lw{2,1}=reshape(w2,outputnum,hiddennum);
net.b{1}=reshape(B1,hiddennum,1);
net.b{2}=B2;

net.trainParam.epochs=100;
net.trainParam.lr=0.1;

[net,per2]=train(net,inputn,outputn);

inputn_test=mapminmax('apply',input_test,inputps);
an=sim(net,inputn_test);
test_simu=mapminmax('reverse',an,outputps);
error=test_simu-output_test;
```

%适应度函数

```
function error =  
fun(x,inputnum,hiddennum,outputnum,net,inputn,outputn)  
  
    w1=x(1:inputnum*hiddennum);  
  
    B1=x(inputnum*hiddennum+1:inputnum*hiddennum+hiddennum);  
  
    w2=x(inputnum*hiddennum+hiddennum+1:inputnum*hiddennum+hiddennum+  
hiddennum*outputnum);  
  
    B2=x(inputnum*hiddennum+hiddennum+hiddennum*outputnum+1:inputnum*  
hiddennum+hiddennum+hiddennum*outputnum+outputnum);  
  
  
    net=newff(inputn,outputn,hiddennum);  
  
    net.trainParam.epochs=20;  
  
    net.trainParam.lr=0.1;  
  
    net.trainParam.goal=0.00001;  
  
    net.trainParam.show=100;  
  
    net.trainParam.showWindow=0;  
  
  
    net.iw{1,1}=reshape(w1,hiddennum,inputnum);  
  
    net.lw{2,1}=reshape(w2,outputnum,hiddennum);  
  
    net.b{1}=reshape(B1,hiddennum,1);  
  
    net.b{2}=B2;  
  
  
    net=train(net,inputn,outputn);  
  
  
    an=sim(net,inputn);  
  
  
    error=sum(abs(an-outputn));
```

%完成变异操作

```

function
ret=Mutation(pmutation,lenchrom,chrom,sizepop,num,maxgen,bound)

    for i=1:sizepop
        pick=rand;
        while pick==0
            pick=rand;
        end
        index=ceil(pick*sizepop);
        pick=rand;
        if pick>pmutation
            continue;
        end
        flag=0;
        while flag==0

            pick=rand;
            while pick==0
                pick=rand;
            end
            pos=ceil(pick*sum(lenchrom));

            pick=rand;
            fg=(rand*(1-num/maxgen))^2;
            if pick>0.5
                chrom(i,pos)=chrom(i,pos)+(bound(pos,2)-chrom(i,pos))*fg;
            else
                chrom(i,pos)=chrom(i,pos)-(chrom(i,pos)-bound(pos,1))*fg;
            end
            flag=test(lenchrom,bound,chrom(i,:));
        end
    end
    ret=chrom;

```

`%测试`

```
function flag=test(lenchrom,bound,code)
x=code;
flag=1;
end
```

`%染色体选择操作`

```
function ret=select(individuals,sizepop)
fitness1=10./individuals.fitness;

sumfitness=sum(fitness1);
sumf=fitness1./sumfitness;

index=[];
for i=1:sizepop
pick=rand;
while pick==0
pick=rand;
end
for i=1:sizepop
pick=pick-sumf(i);
```

```

    if pick<0
        index=[index i];
        break;
    end
end
end

individuals.chrom=individuals.chrom(index,:);
individuals.fitness=individuals.fitness(index);
ret=individuals;

%染色体解码
function ret=Decode(lenchrom,bound,code,opts)
switch opts
case 'binary'% binary coding
    for i=length(lenchrom):-1:1
        data(i)=bitand(code,2^lenchrom(i)-1);
        code=(code-data(i))/(2^lenchrom(i));
    end

    ret=bound(:,1)'+data./(2.^lenchrom-1).*(bound(:,2)-bound(:,1));

case 'grey'

    for i=sum(lenchrom):-1:2

code=bitset(code,i-1,bitxor(bitget(code,i),bitget(code,i-1)));
    end

    for i=length(lenchrom):-1:1
        data(i)=bitand(code,2^lenchrom(i)-1);

```

```

        code=(code-data(i))/(2^lenchrom(i));
    end

ret=bound(:,1)'+data./(2.^lenchrom-1).*(bound(:,2)-bound(:,1))';

    case'float'
        ret=code;
    end

```

%染色体交叉

```

function ret=Cross(pcross,lenchrom,chrom,sizepop,bound
for i=1:sizepop
    pick=rand(1,2);
    while prod(pick)==0
        pick=rand(1,2);
    end
    index=ceil(pick.*sizepop);
    pick=rand;
    while pick==0
        pick=rand;
    end
    if pick>pcross
        continue;
    end
    flag=0;
    while flag==0
        pick=rand;

```

```

while pick==0
pick=rand;
end
pos=ceil(pick.*sum(lenchrom));
pick=rand;
    v1=chrom(index(1),pos);
    v2=chrom(index(2),pos);
chrom(index(1),pos)=pick*v2+(1-pick)*v1;
chrom(index(2),pos)=pick*v1+(1-pick)*v2;
    flag1=test(lenchrom,bound,chrom(index(1),:));
    flag2=test(lenchrom,bound,chrom(index(2),:));
if flag1*flag2==0
flag=0;
else flag=1;
end
end
end
end
ret=chrom;

```

%染色体编码

```

function ret=Code(lenchrom,bound)
flag=0;
while flag==0
pick=rand(1,length(lenchrom));
    ret=bound(:,1)'+(bound(:,2)-bound(:,1)).*pick;
flag=test(lenchrom,bound,ret);
end

```

```
%小波神经网络主函数

M=size(input,2);
N=size(output,2);

n=14;
lr1=0.01;
lr2=0.001;
maxgen=120;

Wjk=randn(n,M);Wjk_1=Wjk;Wjk_2=Wjk_1;
Wij=randn(N,n);Wij_1=Wij;Wij_2=Wij_1;
a=randn(1,n);a_1=a;a_2=a_1;
b=randn(1,n);b_1=b;b_2=b_1;

y=zeros(1,N);
net=zeros(1,n);
net_ab=zeros(1,n);

d_Wjk=zeros(n,M);
d_Wij=zeros(N,n);
d_a=zeros(1,n);
d_b=zeros(1,n);

[inputn,inputps]=mapminmax(input');
```

```

[outputn,outputps]=mapminmax(output');
inputn=inputn';
outputn=outputn';

for i=1:maxgen

error(i)=0;

for kk=1:size(input,1)
    x=inputn(kk,:);
    yqw=outputn(kk,:);

    for j=1:n
        for k=1:M
            net(j)=net(j)+Wjk(j,k)*x(k);
            net_ab(j)=(net(j)-b(j))/a(j);
        end
        temp=mymorlet(net_ab(j));
        for k=1:N
            y=y+Wij(k,j)*temp;
        end
    end

    error(i)=error(i)+sum(abs(yqw-y));

    for j=1:n
        temp=mymorlet(net_ab(j));
        for k=1:N
            d_Wij(k,j)=d_Wij(k,j)-(yqw(k)-y(k))*temp;
        end
        temp=d_mymorlet(net_ab(j));
        for k=1:M

```

```

for l=1:N
    d_Wjk(j,k)=d_Wjk(j,k)+(yqw(l)-y(l))*Wij(l,j) ;
end
d_Wjk(j,k)=-d_Wjk(j,k)*temp*x(k)/a(j);
end
for k=1:N
    d_b(j)=d_b(j)+(yqw(k)-y(k))*Wij(k,j);
end
d_b(j)=d_b(j)*temp/a(j);
%14EËä_d_a
for k=1:N
    d_a(j)=d_a(j)+(yqw(k)-y(k))*Wij(k,j);
end
d_a(j)=d_a(j)*temp*((net(j)-b(j))/b(j))/a(j);
end

Wij=Wij-lr1*d_Wij;
Wjk=Wjk-lr1*d_Wjk;
    b=b-lr2*d_b;
    a=a-lr2*d_a;

d_Wjk=zeros(n,M);
d_Wij=zeros(N,n);
d_a=zeros(1,n);
d_b=zeros(1,n);

    y=zeros(1,N);
net=zeros(1,n);
net_ab=zeros(1,n);

    Wjk_1=Wjk;Wjk_2=Wjk_1;
    Wij_1=Wij;Wij_2=Wij_1;

```

```

        a_1=a;a_2=a_1;

        b_1=b;b_2=b_1;

end

end

x=mapminmax('apply',input_test',inputtps);

x=x';

for i=1:96

x_test=x(i,:);

for j=1:1:n
for k=1:1:M
net(j)=net(j)+Wjk(j,k)*x_test(k);
net_ab(j)=(net(j)-b(j))/a(j);
end
temp=mymorlet(net_ab(j));
for k=1:N
y(k)=y(k)+Wij(k,j)*temp ;
end
end

yuce(i)=y(k);

y=zeros(1,N);
net=zeros(1,n);
net_ab=zeros(1,n);
end

ynn=mapminmax('reverse',yuce,outputtps);

figure(1)
plot(ynn,'r*:')
holdon

```

```

plot(output_test,'bo--')
title('Ô²â¹|ÂÊ','fontsize',12)
legend('Ô²â¹|ÂÊ','Êµ¹Ê¹|ÂÊ')
xlabel('Ê±âµã')
ylabel('·µÇ¹|ÂÊ')

```

%小波神经网络附属程序

```

function y=d_mymorlet(t)

y = -1.75*sin(1.75*t).*exp(-(t.^2)/2)-t*
cos(1.75*t).*exp(-(t.^2)/2) ;

```

%卡曼滤波

```

tic;
closeall;
clear;
clc;
formatcompact;
loadchapter14_sh.mat;

[m,n] = size(sh);

```

```

ts = sh(2:m,1);
tsx = sh(1:m-1,:);

figure;
plot(ts, 'LineWidth', 2);
title('ÉÏÖð,ÊýµÄÄ¿ÈÖªÀÊÝ(1990.12.20-2009.08.19)', 'FontSize', 12);
);
xlabel('»Ö×ÈÖÏÊÝ(1990.12.19-2009.08.19)', 'FontSize', 12);
ylabel('ªÀÊÝ', 'FontSize', 12);
grid on;

ts = ts';
tsx = tsx';

[TS, TSps] = mapminmax(ts, 1, 2);

figure;
plot(TS, 'LineWidth', 2);
title('Ö-ÊÉÏÖð,ÊýµÄÄ¿ÈÖªÀÊÝ¹éÖ»»°óµÄÍñ', 'FontSize', 12);
xlabel('»Ö×ÈÖÏÊÝ(1990.12.19-2009.08.19)', 'FontSize', 12);
ylabel('¹éÖ»»°óµÄªÀÊÝ', 'FontSize', 12);
grid on;
TS = TS';

[TSX, TSXps] = mapminmax(tsx, 1, 2);
TSX = TSX';

[bestmse, bestc, bestg] = SVMcgForRegress(TS, TSX, -8, 8, -8, 8);

disp('´òÓ;ÖÂÑ;Ñª¹û');
str = sprintf('Best Cross Validation MSE = %g Best c = %g Best g = %g', bestmse, bestc, bestg);
disp(str);

```

```

[bestmse,bestc,bestg] =
SVMcgForRegress(TS,TSX,-4,4,-4,4,3,0.5,0.5,0.05);

disp('Best Cross Validation MSE = %g Best c = %g Best g
= %g',bestmse,bestc,bestg);

disp(str);

cmd = ['-c ', num2str(bestc), ' -g ', num2str(bestg) , ' -s 3 -p 0.01'];
model = svmtrain(TS,TSX,cmd);

[predict,mse] = svmpredict(TS,TSX,model);
predict = mapminmax('reverse',predict',TSps);
predict = predict';

str = sprintf('Best Cross Validation MSE = %g Best c = %g Best g
= %g %g %g',mse(2),mse(3)*100);
disp(str);

figure;
hold on;
plot(ts, '-o');
plot(predict, 'r-^');
legend('Observed', 'Predicted');
hold off;

title('Observed vs Predicted', 'FontSize', 12);
xlabel('Time (1990.12.19-2009.08.19)', 'FontSize', 12);
ylabel('Error', 'FontSize', 12);
grid on;

figure;
error = predict - ts';

```

```

end

if nargin < 3
    cmax = 8;
    cmin = -8;
end

[X,Y] = meshgrid(cmin:ctest:cmax,gmin:gstep:gmax);
[m,n] = size(X);
cg = zeros(m,n);

eps = 10^(-4);

bestc = 0;
bestg = 0;
mse = Inf;
basenum = 2;
for i = 1:m
    for j = 1:n
        cmd = ['-v ',num2str(v),' -c ',num2str( basenum^X(i,j) ),' -g ',num2str( basenum^Y(i,j) ),' -s 3 -p 0.1'];
        cg(i,j) = svmtrain(train_label, train, cmd);

        if cg(i,j) < mse
            mse = cg(i,j);
            bestc = basenum^X(i,j);
            bestg = basenum^Y(i,j);
        end

        if abs( cg(i,j)-mse ) <= eps && bestc > basenum^X(i,j)
            mse = cg(i,j);
            bestc = basenum^X(i,j);
            bestg = basenum^Y(i,j);
        end
    end
end

```

```

end

end

[cg,ps] = mapminmax(cg,0,1);
figure;
[C,h] = contour(X,Y,cg,0:msestep:0.5);
clabel(C,h,'FontSize',10,'Color','r');
xlabel('log2c','FontSize',12);
ylabel('log2g','FontSize',12);
firstline = 'SVR² ÆËÛŒ;Ôñ¼á¹ûí¼(µÈ,ßİßÍ¼) [GridSearchMethod]';
secondline = ['Best c=',num2str(bestc), ' g=',num2str(bestg), ...
' CVMse=',num2str(mse)];
title({firstline;secondline},'FontSize',12);
gridon;

figure;
meshc(X,Y,cg);
% mesh(X,Y,cg);
% surf(X,Y,cg);
axis([cmin,cmax,gmin,gmax,0,1]);
xlabel('log2c','FontSize',12);
ylabel('log2g','FontSize',12);
zlabel('MSE','FontSize',12);
firstline = 'SVR² ÆËÛŒ;Ôñ¼á¹ûí¼(3DEÓÍ¼) [GridSearchMethod]';
secondline = ['Best c=',num2str(bestc), ' g=',num2str(bestg), ...
' CVMse=',num2str(mse)];
title({firstline;secondline},'FontSize',12);

```

```

%卡曼滤波函数

function [mse,bestc,bestg] =
SVMcgForRegress(train_label,train,cmin,cmax,gmin,gmax,v,cstep,gstep,~
)

if nargin < 8
    cstep = 0.8;
    gstep = 0.8;
end
if nargin < 7
    v = 5;
end
if nargin < 5
    gmax = 8;
    gmin = -8;
end
if nargin < 3
    cmax = 8;
    cmin = -8;
end

[X,Y] = meshgrid(cmin:cstep:cmax,gmin:gstep:gmax);
[m,n] = size(X);
cg = zeros(m,n);

eps = 10^(-4);

bestc = 0;
bestg = 0;
mse = Inf;
basenum = 2;
for i = 1:m

```

```

    for j = 1:n

        cmd = ['-v ',num2str(v),' -c ',num2str( basenum^X(i,j) ),' -g ',num2str( basenum^Y(i,j) ),' -s 3 -p 0.1'];

        cg(i,j) = svmtrain(train_label, train, cmd);

        if cg(i,j) < mse
            mse = cg(i,j);
            bestc = basenum^X(i,j);
            bestg = basenum^Y(i,j);
        end

        if abs( cg(i,j)-mse ) <= eps && bestc > basenum^X(i,j)
            mse = cg(i,j);
            bestc = basenum^X(i,j);
            bestg = basenum^Y(i,j);
        end
    end
end
end
end

```